

TER L3 MIASHS 2025 - 2026

Sujet - Web scraping pour enrichir une base de données

Rapport final pour le 10/04/2026

*Rédigé par Emmy BERNARD, Anaëlle EON, Youna-Marie BADOUARD,
Juliette ORZAKIEWICZ*

Encadré par Laurent Ughetto

Sommaire

Sommaire.....	2
Introduction.....	3
Description et appropriation du sujet.....	3
I - Partie organisationnelle.....	5
1) Planification et organisation.....	5
2) Temps de travail total effectif, ainsi qu'une analyse des décalages éventuels par rapport au planning initial.....	5
II - Partie technique.....	7
1) Test des méthodes de web scraping.....	7
a) Scrapy.....	7
b) Requests et BeautifulSoup 4.....	7
c) Sélénium.....	7
2) Structure de la base de données.....	8
3) Les tables.....	11
a) Driver.....	11
b) Team.....	13
c) Season.....	14
d) Circuit.....	16
e) Grand prix.....	17
f) Standings.....	19
g) RaceDriver.....	20
4) Les problèmes communs rencontrés.....	22
a) Wikipédia.....	22
b) L' API Ergast.....	24
5) Les finitions à apporter.....	24
III - Prise de recul et perspectives.....	25
Conclusion.....	25
Sitographie et Bibliographie.....	27
Annexe.....	30

Introduction

Ce rapport constitue le compte rendu final de notre Travail Encadré de Recherche (TER), consacré à l'acquisition de données par web scraping pour l'enrichissement d'une base de données dédiée à la Formule 1, utilisée dans un cours de bases de données en Master.

Une saison de Formule 1 se déroule sur une année, de mars à décembre, tout autour du monde. Chaque Grand Prix se passe sur un week-end, accueilli par un circuit. Un système de points donne lieu au championnat du monde, le pilote avec le plus de points gagne. De même pour les écuries : avec les points cumulés de leurs pilotes, celui qui en a le plus à la fin gagne le championnat constructeur. Ce domaine d'application représente un volume important de données historique depuis 1950, date du premier Championnat du monde de Formule 1.

Notre projet est réalisé grâce au web scraping, qui désigne l'ensemble des techniques permettant d'extraire automatiquement des données structurées à partir de sites web. Contrairement à une collecte manuelle, ce procédé utilise des scripts informatiques pour parcourir le code HTML des pages, cibler des informations précises et les organiser dans un format exploitable (comme une base de données ou un fichier CSV).

Ce document s'articule autour de deux axes majeurs : la partie organisationnelle et la partie technique. Nous restituerons d'abord le sujet et comment nous nous le sommes approprié, avant de détailler les axes principaux. Dans la première partie, nous aborderons notre organisation générale sur le semestre, puis la comparaison avec notre rétroplanning fait en début de semestre. Ensuite, dans la seconde partie nous détaillerons la description de la base de données et comment nous l'avons construite. Enfin, les problèmes communs rencontrés et la prise de recul seront expliqués en dernier.

Description et appropriation du sujet

Notre sujet s'inscrit dans le cadre du module de Bases de Données du Master 1 MAS et porte sur l'enrichissement d'une base de données préexistante. Le jeu de données initial couvre une période spécifique allant de janvier 2013 à septembre 2014, soit une saison et demie de Formule 1, tout en incluant un historique des pilotes champions du monde depuis 1950. L'objectif principal du TER est de compléter les sept tables en regroupant et organisant les données à propos de la Formule 1 (*Team, Driver, Standings, Circuit, Grandprix, Racedriver, Testdriver*) en intégrant des données manquantes ou en affinant la structure actuelle.

La confrontation avec la réalité de la collecte de données a nécessité une adaptation stratégique de la base de données. En effet, certaines informations s'avèrent soit inaccessibles, soit trop complexes à extraire dans le temps imparti. Suite à des réunions avec notre référent, nous avons pris la liberté de modifier la base de données : certaines colonnes ont été retirées tandis que d'autres ont été ajoutées pour exploiter des informations disponibles mais non prévues initialement.

Pour mener à bien cet enrichissement, il est essentiel de maîtriser les concepts techniques suivants :

Sites avec ou sans Javascript : La stratégie d'extraction dépend de la nature du site cible. Les sites sans Javascript délivrent un code HTML complet où les données sont directement accessibles dans le code source. À l'inverse, lorsque la page contient du code Javascript, celui-ci est exécuté après le chargement de la page, ce qui ajoute du code HTML. Cela nécessite des outils de simulation de navigateur pour accéder à ces données générées par Javascript.

Les API (Application Programming Interface) : Contrairement au scraping qui "lit" l'interface visuelle, une API permet un échange de données structuré et officiel entre deux applications. Lorsqu'elles sont disponibles, les API constituent une source plus fiable que le scraping, car elles fournissent directement les données au format JSON ou XML sans nécessiter d'analyse de code source HTML complexe.

Les Bibliothèques Python : C'est un ensemble de modules et de fonctions pré écrites qui permettent de réaliser directement des tâches spécifiques, sans avoir à développer chaque fonctionnalité. Par exemple, elles nous servent à récupérer le code source d'une page web et extraire les données de cette dernière. Il est nécessaire de les importer au début de notre code afin de pouvoir les utiliser.

La Base de Données : Il s'agit d'un système structuré (ici relationnel) organisé en tables liées par des clés étrangères (référence à une autre table). La base de données permet de stocker et de consulter facilement un grand nombre de données.

I - Partie organisationnelle

1) Planification et organisation

Afin de mener notre projet à bien, nous avons dû mettre en place une organisation rigoureuse, et ce dès le début du semestre. Le défi principal était de se répartir les tâches afin d'optimiser notre temps de travail, sans être à quatre sur une même tâche. Nous avons donc dû nous faire confiance pour avancer comme nous voulions dans le projet.

Nous avons opté pour la méthode suivante : l'idée est, plutôt que de vouloir construire toute la base de données en même temps, de procéder table par table, puis de faire les liens. Comme nous n'avions que notre script à la fois, cette approche nous a permis de le tester sur des échantillons réduits avant de les généraliser jusqu'en 1950.

Afin de maintenir une communication fluide entre nous, nous échangeons via un groupe sur les réseaux sociaux. Grâce à cela, nous étions certaines qu'au moins l'une d'entre nous verrait le message afin d'apporter son aide.

Par ailleurs, nous avons décidé de mettre en place deux réunions hebdomadaires. Une réunion avait lieu tous les mercredis matins en présence de notre référent afin de faire le point sur notre avancement et de poser nos éventuelles questions. En parallèle, nous préparions toutes les 4 cette réunion la veille au soir, sur notre temps personnel (de 18h à 20h).

Pour finir, notre professeur référent a ouvert un espace cours sur Coursus (Figure 1) à laquelle nous étions les seuls à avoir accès. Grâce à cela, nous avons pu centraliser tous nos documents et nos codes afin que tout le monde puisse les consulter facilement. Chaque semaine, nous y ajoutions également un compte-rendu, permettant à chacune de se remémorer les décisions prises (voir un exemple en Annexe 1).



Figure 1 : Page cursus

2) Temps de travail total effectif, ainsi qu'une analyse des décalages éventuels par rapport au planning initial

Comme évoqué précédemment nous avons réalisé un planning prévisionnel en début de TER (Annexe 2 et 4). À la fin de ce dernier, nous avons refait un autre planning afin de pouvoir comparer les deux, et ainsi voir les décalages éventuels (Annexe 3). En ce qui concerne la phase de conception et de documentation, nous avons suivi nos prévisions initiales. Cependant, c'est devenu plus difficile de tenir le planning lorsque nous sommes passées au codage. En effet, pour certaines tables, le temps de travail a doublé.

Au début du semestre, nous avons estimé la charge de travail globale à environ quarante huit heures par personne. Dans les faits, ce volume a été largement dépassé. En effet, pour les tables nous sommes parties sur une base de douze heures chacune. Cependant certaines tables comme la

table *Team* ou encore *Grandprix* ont nécessité beaucoup plus de temps. La principale raison de cet important décalage pour ces tables, surtout la table *Team*, est qu'il a fallu changer de méthode de scrapping en cours de route. Nous avons donc dû nous familiariser avec cette nouvelle méthode. De plus, les recherches dans la documentation de l'API sont restées dans un premier temps en surface. Cela nous a valu l'implémentation de codes longs et parfois complexes, qui auraient pu être évités.

Par ailleurs, dans notre planning initial, nous avions prévu de créer la base de données SQL si nous avions le temps. Avec le retard accumulé, nous n'avons pas pu effectuer cette tâche.

Bien que notre planning initial ait été bousculé par des imprévus techniques, nous avons tout de même produit des fichiers CSV cohérents contenant les données attendues.

II - Partie technique

1) Test des méthodes de web scraping

Un travail préalable de recherche concernant les différentes méthodes de web scraping nous a menées à en tester quatre : les plus connues et utilisées. Les bibliothèques Requests & BeautifulSoup4 et la méthode scrapy permettent de traiter les sites sans JavaScript, tandis que Selenium s'occupe des pages avec JavaScript. (voir annexes 5, 6 et 7)

a) Scrapy

Pour tester cette méthode, nous avons suivi un tutoriel en ligne permettant de scraper un site d'entraînement dédié. Nous avons exécuté les commandes étape par étape, mais plusieurs d'entre elles ont provoqué des erreurs.

Un projet Scrapy se structure sous la forme d'un dossier contenant plusieurs fichiers de base. Cette architecture nous a semblé complexe : l'utilité de ces fichiers, ainsi que de nombreuses manipulations, nous échappe. De plus, cette méthode impose un travail conséquent via le terminal plutôt que dans un script Python classique, ce dont nous n'avons pas l'habitude. Cette technique permet la récupération d'informations essentiellement sur des sites web sans JavaScript. Or, des méthodes comme Requests et BeautifulSoup, qui remplissent la même fonction, sont bien plus intuitives.

Au vu des autres méthodes testées et de la prise en main difficile de Scrapy, outil que nous ne maîtrisons toujours pas malgré l'aide de l'intelligence artificielle, nous avons décidé d'abandonner cette piste au profit de solutions plus accessibles et mieux adaptées à nos besoins.

b) Requests et BeautifulSoup 4

En ce qui concerne les méthodes BeautifulSoup4 et Requests, elles ont été les méthodes les plus "simples" à reprendre en main car nous avons déjà eu des cours de scraping où nous les utilisons.

Un rappel a tout de même été nécessaire : nous nous sommes documentées à nouveau et avons relu quelques-uns de nos anciens codes.

Comme la prise en main a été assez rapide, nous avons directement commencé à coder la table *Driver*. Nous sommes rapidement parvenues à scraper les premières informations.

c) Sélénium

L'extraction de données sur des sites web dont le contenu est généré via JavaScript rend l'utilisation de bibliothèques classiques comme BeautifulSoup ou Requests inefficace. Pour pallier cette limite, nous avons exploré la méthode Selenium.

Après avoir suivi des tutoriels d'introduction, nous avons surmonté les premières difficultés liées à la configuration de l'environnement et aux importations nécessaires. Pour nous familiariser

avec cet outil, nous avons réalisé des tests préliminaires sur la page web YouTube, qui était le site utilisé dans le tutoriel, afin de maîtriser les commandes de base, avant de les appliquer aux sites cibles de notre projet.

Finalement, nous avons choisi d'abandonner cette approche à tort. La complexité de l'outil et son utilisation ont été jugées excessives par rapport à l'utilité réelle estimée de Selenium pour les sources dont nous disposions.

2) Structure de la base de données

La base de données initiale se présente sous la forme suivante (voir tableau et schéma ci-après). Elle est constituée des sept tables : *Driver*, *Grandprix*, *Circuit*, *Team*, *Racedriver*, *Testdriver* et *Standings*, et concerne les saisons 2013 et 2014 de Formule 1 (voir annexe 8).

Durant nos douze semaines de projet, nous avons été invitées à réfléchir sur la forme et le fond de ces tables. De plus, nous avons été régulièrement confrontées à divers problèmes de modélisation dus aux règlements changeants de la Formule 1 au cours des années. Nous avons la possibilité d'y apporter des modifications, comme ajouter ou supprimer une table, ou encore modifier des attributs. Nous devons garder à l'esprit que cette base de données s'adressait à un public de Master 1 MAS et qu'elle était créée à des fins pédagogiques. La limite haute du nombre de tables constituant cette base a été fixée à dix.

Cette réflexion a également été menée préalablement par notre référent, qui avait des demandes précises de modification.

Premièrement, dans la table *Grandprix*, l'identifiant n'était pas "dépourvu de sens" (clé technique). Il était au format « YYYYRR », où YYYY représentait l'année du Grand Prix et RR son rang dans la saison en cours. C'était un défaut de conception, car les élèves pouvaient extraire de cet identifiant une information qui ne devrait pas y être dédiée. La structure devait donc être modifiée à la demande de notre référent.

Nous avons ajouté la table *Season*. Cette dernière permet de faire le lien avec de nombreuses tables évoquant la temporalité (*Grandprix*, *Team* et *Racedriver*). De plus, elle récolte des informations globales sur la saison, demandées explicitement :

- | | |
|-------------------------------------|--|
| - Le nom du champion de la saison ; | - Un booléen (<i>True/False</i>) indiquant si, |
| - Le nom du manufacturier de pneus | durant la saison, des numéros fixes |
| lorsqu'il est commun à toutes les | étaient attribués (pour la saison ou |
| écuries ; | pour la carrière du pilote). |

Grâce à cette nouvelle table centrale, l'identifiant de la table *Grandprix* a été modifié en un identifiant classique auto-incrémenté. La saison est devenue une clé étrangère dans les trois tables citées précédemment : *Team* (avec les attributs *tstart* et *tend* permettant la modélisation d'une période d'activité), *Racedriver* (faisant partie de la clé primaire) et Grand Prix.

Parmi les autres modifications demandées, nous devons intégrer le numéro permanent des pilotes dans la table *Driver*. Après constat du fonctionnement réel (numéros changeant selon les années, par exemple) et des informations disponibles, le numéro se trouve désormais dans la table *Racedriver*, ce qui permet de mettre en avant le numéro du pilote en fonction de l'équipe et de la saison.

Ensuite, comme en témoignent les deux schémas de tables (Figure 2 et Figure 3), nous avons pris plusieurs initiatives personnelles.

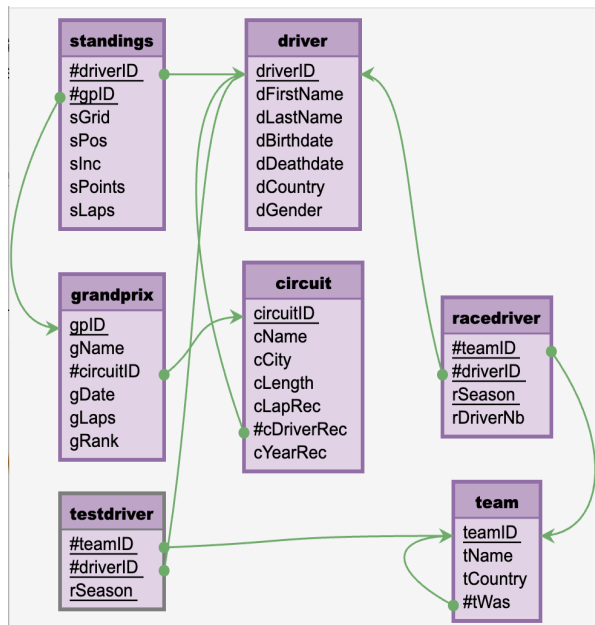


Figure 2 : Base de données initiale

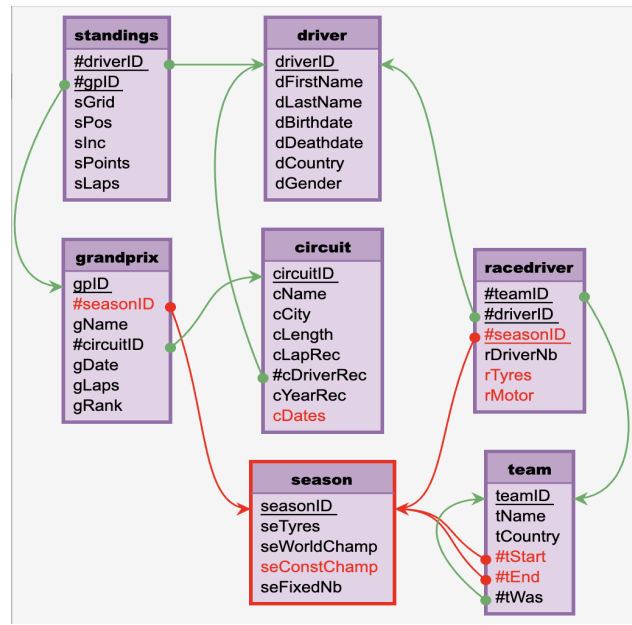


Figure 3 : Base de données finale, modifications en rouge

La table *Circuit* comprend désormais une information supplémentaire permettant de répertorier tous les circuits : l'attribut « dates » comportant parfois des périodes, parfois des dates isolées, détaille les "versions" des circuits ayant le même nom.

Dans la table *Season*, nous nous sommes également permis un ajout qui nous paraissait intéressant : l'attribut « seConstChamp », qui indique, lorsqu'il existe (à partir de 1958), le constructeur sacré champion de la saison. En effet, la Formule 1 est aussi une compétition mécanique et technologique.

À cela s'ajoute une modification du format des tables *Team* et *Racedriver*, qui sont étroitement liées. Suite à nos recherches, le format initial de la base de données ne permettait pas de rendre compte de situations ayant eu lieu historiquement, notamment entre 1950 et 1973. Durant cette période, pour une même saison, un même pilote et une même équipe, le numéro de la voiture attribué pouvait changer à chaque Grand Prix. C'est une spécificité qui n'avait pas été appréhendée lors de la conception initiale de la base, laquelle ne concernait que les saisons 2013 et 2014. Pour pallier ce problème, nous avons délibérément isolé cette période complexe pour ne conserver que les saisons à partir de 1974, dont la représentation est plus simple. Il aurait fallu, sinon, ajouter l'identifiant du Grand Prix à la clé primaire (déjà composée des identifiants du pilote,

de son équipe et de la saison). Or, cette information aurait été inutile pour les années suivantes, causant une redondance d'information importante.

Parallèlement à cette simplification temporelle, la structure de la table *Team* a évolué lors de la semaine 5. Notre référent nous a demandé d'intégrer le nom du motoriste et du manufacturier de pneus, ainsi que les dates de début et de fin d'activité de l'écurie. Pour ces dernières, nous avons instauré une règle précise : si l'équipe est toujours actuelle, on renvoie NULL, et si elle n'a existé qu'un an ou moins, la date de fin est égale à la date de début.

Une colonne localisation avait également été ajoutée pour différencier le pays de création du lieu du siège social, car l'infobox Wikipédia affichait souvent des données divergentes. Cependant, en changeant de source pour StatsF1, cette mention a disparu, nous poussant à supprimer cette colonne.

Enfin, les attributs concernant les manufacturiers de pneus et de moteurs, initialement prévus dans la table *Team*, ont finalement été intégrés à la table *Racedriver*. Ce choix est justifié par un manque d'informations centralisées au niveau des écuries, et surtout par les changements pouvant survenir durant la période d'activité d'une écurie (entre *tstart* et *tend*) que nous ne pouvions pas modéliser autrement.

Ce changement de stratégie s'explique aussi par l'évolution de nos méthodes de récupération : alors que nous utilisions initialement Requests et BeautifulSoup4 (site web sans JavaScript), nous sommes passés sur Sélénium (site web avec JavaScript). Nous nous sommes rendu compte que les informations n'étaient pas directement sur la page principale de l'écurie, mais que le scraping effectué pour la table *Racedriver* permettait de les récupérer beaucoup plus facilement. C'était également plus pertinent car *Racedriver* permet d'associer précisément un moteur et un pneu à chaque année de l'écurie.

Toutefois, ce choix n'est pas forcément optimal : il provoque des lignes en « double » dans la table *Racedriver*, car ces informations sont identiques pour les différents pilotes d'une même écurie, comme l'illustre la Figure 4.

#teamID	#driverID	#seasonID	rDriverNb	rTyres	rMotor
117	510	1974	1	Goodyear	Ford Cosworth
117	333	1974	2	Goodyear	Ford Cosworth

Figure 4 : Exemple de redondance d'information dans la table *Racedriver*

Enfin, nous pouvons remarquer qu'une table est manquante : *Testdriver*. Cela est dû à un manque de temps. Aussi, tout ce qui a été présenté précédemment constitue un idéal à atteindre. À la fin de notre processus de recherche, certaines tables restent en suspens, avec des jointures non abouties ou comportant certaines erreurs, ainsi que des problèmes persistants propres à chaque table. De plus, la langue utilisée pour récupérer certaines informations n'est pas homogène (anglais, langue du pays ou français), cela est provoqué par le choix du site scrapé. Tout cela est détaillé, table par table, dans la suite de ce rapport (exemple des tables obtenues dans l'annexe 9).

3) Les tables

Afin de récupérer nos données, nous avons plusieurs sites web à disposition, dont Statsf1, un site web de référence en Formule 1, amateur, dont la structure est rigoureuse et uniforme. Il y a également Wikipédia, le site officiel de Formula 1 et une API, où les informations sont dans ce cas là déjà en format uniforme et accessibles facilement.

Dans un premier temps, nous nous sommes restreintes à l'utilisation de Requests et BeautifulSoup 4. Cela nous a limitées à des sites web sans JavaScript comme Wikipédia. Cependant, pour certaines tables, les sites avec JavaScript sont essentiels. Nous nous sommes donc rapidement formées à Selenium grâce à l'intelligence artificielle (voir annexe 10).

a) Driver

En ce qui concerne la table *Driver*, nous avons réussi à compiler un document csv propre contenant :

- L'identifiant du pilote;
- son prénom;
- son nom de famille;
- sa date de naissance;
- sa date de décès;
- son pays d'origine;
- son genre.

Finalement, nous avons répertorié plus de 700 pilotes.

Les différentes sources

Afin de trouver ces informations, nous avons comparé plusieurs sources.

- Stats F1

C'est le site amateur de référence en F1. Ce dernier nous a semblé être un choix pertinent car il répertorie les pilotes depuis 1950. Cependant, la structure du site nécessite l'utilisation de Sélénium, alors que nous utilisons Requests et BeautifulSoup 4.

- L'API ergast

Elle est très précise pour les statistiques de course, mais est plus limitée pour les détails biographiques "humains" des pilotes. Par exemple, nous n'avons pas d'informations sur les potentiels décès des pilotes.

- Wikipédia

Wikipédia est venu plutôt naturellement ensuite car c'est structuré toujours de la même manière et c'est très facile à comprendre. En effet, les "Infobox" (les petits tableaux à droite des pages, voir Figure 5) sont standardisées, malgré quelques variations, ce qui facilite énormément le travail de l'ordinateur pour repérer où se trouvent les informations que nous avons à récupérer. Voici un exemple avec la page de Charles Leclerc.



The image shows the Wikipedia infobox for Charles Leclerc. It includes a photo of him in a red Ferrari racing suit, followed by a table with his biographical and career details. The table is organized into sections: Biographie, Carrière, and Statistiques.

Charles Leclerc		
		
Charles Leclerc en 2026.		
Biographie		
Nom complet	Charles Marc Hervé Perceval Leclerc	
Surnom	Il Predestinato, Lord Perceval	
Date de naissance	16 octobre 1997 (28 ans)	
Lieu de naissance	Monte-Carlo, Monaco	
Nationalité	Monégasque	
Site web	charlesleclerc.com	
Carrière		
Années d'activité	Depuis 2014	
Qualité	Pilote automobile	
Parcours		
Années	Écurie	C. (V.)
2018	 Sauber	21 (0)
2019-	 Ferrari	153 (8)
Statistiques		
Numéro permanent	16	
Nombre de courses	174	
Pole positions	27	
Meilleurs tours en course	11	
Podiums	52	
Victoires	8	
Meilleur classement	2022	
modifier		

Figure 5 : Infobox de la page de Charles Leclerc

Afin d'accéder aux pages de tous les pilotes, nous avons réalisé une boucle à partir d'une page Wikipédia (Figure 6) recensant tous les pilotes de F1. En effet, nous avons trouvé une page Wikipédia qui répertorie tous les noms des pilotes (du moins c'est ce que nous pensions).

-  [George Abecassis](#) : 2 Grands Prix (1951-1952)
-  [Kenny Acheson](#) : 3 Grands Prix (1983, 1985)
-  [Philippe Adams](#) : 2 Grands Prix (1994)
-  [Walt Ader](#) : 1 Indy 500 (1950)

Figure 6 : Page wikipédia des pilotes de F1

Les noms des pilotes sont en fait des liens cliquables qui emmènent directement sur la page Wikipédia personnelle du pilote, où nous retrouvons, comme pour Charles Leclerc, l'infobox avec toutes les informations nécessaires.

Les problèmes rencontrés

Pour commencer, sur la page Wikipédia où se trouvent le nom des pilotes, il manque finalement les pilotes femmes, qui ont une autre page Wikipédia pour elles. Étant donné qu'elles sont au nombre de 5, nous avons pris la décision de les rajouter manuellement dans le code.

Aussi, il nous fallait récupérer le genre des pilotes, qui n'était pas renseigné directement dans l'infobox. Grâce à la liste des femmes, nous avons pu renseigner le genre des pilotes grâce à une boucle. L'idée est que si le nom du pilote n'apparaît pas dans la liste des femmes, alors on marque "H" dans sa colonne genre; et à l'inverse, on renseigne "F" dans cette dernière.

De plus, les noms des pilotes internationaux (accents slaves, scandinaves, etc.) sont parfois difficiles à traiter. Selon l'encodage de la page, un même nom peut s'écrire de deux façons différentes, ce qui risque de créer des doublons ou des erreurs de lien avec les autres tables de notre base de données.

Critique des résultats

Nous avons réussi à obtenir une table très équilibrée. Contrairement à nos craintes initiales, les pilotes des années 1950 sont aussi bien documentés que ceux de 2024. Cela nous permet d'avoir une base de données historique cohérente sur toute la durée. Cependant, nous ne récupérons pas la totalité des noms de pilotes F1 ayant existé. En effet, nous souhaitions initialement récupérer également les pilotes d'essai.

b) Team

Concernant la table *Team*, nous avons réussi à fournir un fichier CSV propre contenant :

- | | |
|---------------------|--|
| - Nom de l'écurie; | - Date de fin; |
| - Pays de l'écurie; | - Identifiant de l'écurie; |
| - Date de début; | - Twas (clé étrangère qui renvoie à un identifiant). |

Finalement, nous avons recensé 215 écuries.

Les différentes sources

Pour pouvoir trouver les informations, nous avons comparé plusieurs sites :

- Écuries avec Wikipédia

Sur Wikipédia, nous avons trouvé une page qui liste toutes les écuries de F1 depuis la création du championnat (1950). Cependant, c'est une liste non-exhaustive, elle ne prend pas en compte les écuries privées, ainsi que les écuries ayant participé uniquement aux 500 miles d'Indianapolis (Figure 7). C'est une compétition automobile américaine qui est organisée chaque année, depuis 1911, le lundi précédent le lundi du memorial day. Cette course a parfois été intégrée au championnat de F1.

Les écuries sont listées dans un tableau par ordre alphabétique.

Écurie	Pays	Années en F1	Titres constructeurs 1 ^{er} / 2 ^e / 3 ^e	GP F1 disputés	Victoires
Alex von Falkenhausen Motorenbau	 Allemagne	1952 - 1953	-	4	-
Alfa Romeo	 Italie	1950 - 1985, 2019 - 2023	-	110	10

Figure 7 : Page Wikipédia des écuries de F1

Comme pour les pages des pilotes, les écuries possèdent une infobox. Cependant certaines informations comme le twas, n'apparaissent pas. C'est pourquoi nous nous sommes tournées vers StatsF1.

- Écuries avec StatsF1

Pour StatsF1 qui est le site de référence des amateurs, la structure est assez similaire à celle de Wikipédia, on retrouve un onglet "Constructeurs". Cependant, cette base de données est plus complète, elle recense notamment les écuries qui ont fait les 500 miles d'Indianapolis (Figure 8).

18 constructeurs	Nation	Début en	Vainqueur	Champion du Monde
A-T-S		1963		
Adams		1950		
AFM		1952		
AGS		1986		
Alfa Romeo		1950		
Alfa Special		1963		

Figure 8 : Page de StatsF1 concernant les écuries

Les problèmes rencontrés

Le défi majeur est apparu avec la gestion du *twas*. On le rappelle, le *twas* est une clé étrangère qui doit renvoyer NULL si l'écurie a été créée de toute pièce (ou existe depuis la première saison référencée), ou sinon l'identifiant de l'ancienne équipe si elle a été créée suite au rachat ou à la transformation d'une équipe existante de la base. Le problème est que dans l'infobox des écuries, il n'y a aucune information à ce propos. Le seul endroit où cela est mentionné, c'est dans l'histoire de l'écurie. Les informations sont donc impossibles à coder. Nous avons donc décidé de créer un dictionnaire manuel qui retrace la généalogie de chaque écurie. Le dictionnaire est du type : (écurie fille : écurie mère) (voir annexe 11). Cette méthode est longue, et nous étions conscientes qu'il pouvait y avoir des erreurs. En effet, les informations sur Wikipédia peuvent être incomplètes, la dernière mise à jour date du GP d'Abou Dabi en 2025.

Le code semble fonctionner. Cependant, en vérifiant certains cas, nous nous sommes rendues compte que le *twas* ne pointe pas au bon endroit. En effet, lorsqu'une écurie qui a été rachetée possède plusieurs périodes d'activité, il renvoie à la première occurrence de l'écurie. A ce moment-là, au vu du temps qu'ils nous restait, nous avons gardé ce code en précisant qu'il pouvait comporter des erreurs. Entre-temps, nous sommes passées sur StatsF1. La seule différence notable est que sur la page des écuries, il y a une rubrique "Filiation" (voir annexe 12) qui liste l'historique de ces dernières avec notamment le nom de la précédente écurie ainsi que la période.

c) Season

L'implémentation de la table *Season* nécessite la récupération des informations suivantes :

- Le constructeur champion du monde ;
- Le manufacturier de pneus (lorsque celui-ci est commun à toutes les écuries) ;
- L'attribution ou non d'un numéro permanent (booléen).
- L'année de la saison ;
- Le pilote champion du monde;

Nous avons récupéré les informations concernant les 76 années de Formule 1.

Afin de rassembler ces données, nous avons utilisé de manière complémentaire l'API, ainsi que les pages Wikipédia des saisons en version anglaise et française. En effet, malgré un thème commun, les pages anglophones ne fournissent pas toujours les mêmes informations que les pages françaises, ces dernières n'étant pas de simples traductions.

Les différentes sources

- L'apport de l'API et de Wikipédia Anglais

L'API s'est révélée assez « pauvre » concernant les détails des saisons. Elle nous a essentiellement permis de récupérer l'année et un lien direct vers la page Wikipédia anglaise correspondante (Figure 9). Sur cette version anglophone, le

1950 Formula One season	
Drivers' Champion: Giuseppe Farina	
Previous: 1949	Next: 1951
Races by country · Races by venue	

Figure 9 : Page wikipédia anglaise

contenu reste restreint : nous n'y avons extrait que le nom du pilote champion.

- L'exploitation de la page française

C'est dans un second temps que nous avons sollicité la page française, bien plus fournie (Figure 10). L'automatisation du parcours des saisons est simplifiée par l'utilisation d'une URL uniforme dont seule l'année varie :

"https://fr.Wikipédia.org/wiki/Championnat_du_monde_de_Formule_1_{année}".

Nous avons extrait les informations directement depuis l'infobox. Nous y avons récupéré un maximum de données, même celles non demandées initialement, comme le nombre de manches, le nombre de participants ou d'équipes engagées. Bien que ces informations complémentaires ne figurent pas dans la base de données finale, elles constituent un excellent outil de vérification pour la cohérence des tables *Grandprix* ou *Circuit*.



Championnat du monde de Formule 1 2010	
Généralités	
Sport	Formule 1
Création	1950
Organisateur(s)	FIA
Édition	61 ^e
Participants	27
Nombre de manches	19 Grands Prix
Site web officiel	www.fia.com www.formula1.com
Palmarès	
Champion pilote	 Sebastian Vettel
Champion constructeur	 Red Bull-Renault
Navigation	
◀ 2009	2011 ▶
modifier	

Figure 10 : Page wikipédia française

Les problèmes rencontrés

Suite à cette phase de récupération, certaines données concernant les numéros de pilotes permanents et les manufacturiers de pneus restaient incomplètes. Faute d'un format uniforme, nous avons opté pour un traitement manuel basé sur des pages historiques dédiées.

1. Les numéros permanents : L'historique indique qu'à partir de 2014, les pilotes choisissent un numéro pour l'ensemble de leur carrière. Avant cette date, les numéros étaient attribués selon le classement de la saison précédente. Ainsi, dans notre table, l'attribut « numéro fixe » est défini sur FALSE jusqu'en 2013, puis passe à TRUE à partir de 2014.
2. Les manufacturiers de pneus : Identifier les périodes de monopole a été complexe, car de nombreuses périodes d'activité se recoupent. En comparant les textes officiels et le schéma historique réalisé par un particulier (Figure 11), nous avons identifié les phases (voir annexe 13) où un manufacturier est le seul présent, soit par règlement (monopole imposé), soit « de fait » (retrait des concurrents).

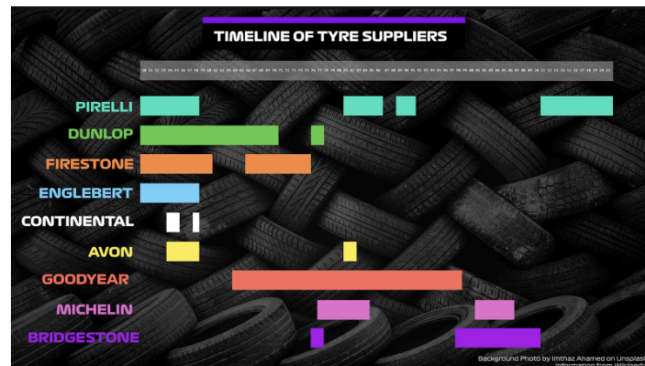


Figure 11 : Schéma d'un particulier à propos des manufacturiers de pneus.

Critique des résultats

Dans notre cas, l'utilisation de l'API et du site anglais aurait pu être remplacée par une simple boucle sur les années dans les pages Wikipédia. Les noms des champions sont aussi présents sur le site français. Cependant, le travail avec l'API a été réalisé en premier et nous l'avons donc conservé. Il

nous a notamment permis de comparer les noms récupérés sur les deux sites, qui se sont avérés identiques.

d) Circuit

Concernant la table *Circuit*, nous avons récupéré les informations suivantes :

- Le nom;
- La ville;
- Le pays;
- La longueur;
- Le record de tour;
- L'année du record;
- Le nom du pilote ayant effectué le record.

Nous récupérons un fichier de 371 circuits, en incluant les variantes.

Les différentes sources

- Stats F1

Après une première étude des sites dédiés à la Formule 1, nous avons initialement extrait les données via StatsF1 pour obtenir les noms des circuits, leurs pays et leurs villes (Figure 12). Toutefois, nous avons constaté plusieurs incohérences : des inversions entre les noms de lieux et les circuits, ainsi que des informations manquantes ou erronées.

78 Circuits		Pays	Début en	Nb
Adelaïde	Adelaide Street Circuit	 Australie	1985	11
Aïda	Okayama International Circuit	 Japon	1994	2
Ain-Diab		 Maroc	1958	1

Figure 12 : Page à propos des circuits sur StatsF1.

De plus, la gestion des records de piste pose un défi technique. Le site permet de récupérer les meilleurs temps par année, ce qui impose de comparer manuellement chaque saison pour isoler le record absolu. Un autre point de vigilance est apparu : ces records sont souvent des records en course. Ils ne reflètent donc pas nécessairement le temps le plus rapide du week-end, qui peut être établi lors des qualifications ou des essais libres.

- Page Wikipédia française

Face à ces difficultés, nous nous sommes tournées vers Wikipédia. Le regroupement des circuits dans un tableau centralisé et toutes leurs informations imbriquées dans une infobox simplifient considérablement le scraping.

Cependant, la version française de Wikipédia (Figure 13) présente deux défauts majeurs. Les records ne sont pas toujours

vérifiés et ne concernent souvent qu'une seule configuration de circuit, ignorant les changements de tracés au fil des ans. De plus, la page française ne répertorie que 76 circuits, alors que StatsF1 et la version anglaise de Wikipédia en comptabilisent 78.

Meilleur tour (1993)	
Temps	1 min 13 s 371
Pilote	Ayrton Senna

Figure 13 : Page wikipédia française.

- [Page Wikipédia anglaise](#)

Nous avons finalement opté pour la page Wikipédia en anglais. En plus d'être plus complète, elle offre l'avantage de détailler les records pour chaque variante historique des circuits. Cette précision garantit une base de données plus représentative de l'histoire des circuits de Formule 1.

- [L'API ergast](#)

Nous avons décidé d'utiliser également l'API afin de gagner du temps sur le scraping d'informations particulièrement difficiles à récupérer sur Wikipédia.

Grand Prix Circuit (2021–present)	
Surface	Asphalt
Length	5.278 km (3.280 mi)
Turns	14
Race lap record	1:19.813 (Charles Leclerc, Ferrari SF-24, 2024, F1)
Grand Prix Circuit (1996–2020)	
Surface	Asphalt
Length	5.303 km (3.295 mi)
Turns	16
Race lap record	1:24.125 (Michael Schumacher, Ferrari F2004, 2004, F1)

Figure 14 : Page wikipédia anglaise.

Lien avec la table *Driver*

Le lien avec la table *Driver*, donc ici la récupération du l'identifiant du pilote, se fait sans problème. A l'exception de quelques noms de pilotes qui ne sont pas orthographiés de la même façon dans les deux tables, et qui ne sont donc pas récupérés.

e) Grand prix

L'implémentation de la table *Grandprix* demandait de récupérer les informations suivantes :

- Le nom du grand prix;
- Le rang du grand prix dans la saison;
- Le circuit utilisé;
- Le nombre de tour à réaliser;
- La saison (année du grand prix).

Nous récupérons un fichier contenant 1 174 grands prix.

Les différentes sources

- [Le site Wikipédia des grands prix](#)

Nous retrouvons sur cette page un tableau contenant le nom du grand prix, avec un lien cliquable vers la page générale de ce dernier. Or, elle a été abandonnée car elle ne contient pas les informations nécessaires notamment à propos des circuits associées.

- [Les sites Wikipédia des saisons](#)

Sur Wikipédia, chaque saison de formule 1 possède sa propre page avec la liste des grands prix ayant eu lieu, le rang dans la saison et le lien vers le grand prix. Ce qui a promu cette solution est le parcours simple de chaque saison, étant donné que les liens des pages Wikipédia sont uniformes : "[https://en.Wikipédia.org/wiki/\[annéesouhaitée\] Formula One World Championship](https://en.Wikipédia.org/wiki/[annéesouhaitée] Formula One World Championship)".

Nous pouvons alors, en changeant seulement [annéesouhaitée], parcourir chaque page Wikipédia qui semblent avoir une certaine homogénéité avec la présence systématique de ce tableau (Figure 15) :

Cette méthode ne se suffisait pas à elle-même car l'information concernant le nombre de tours à effectuer lors de chaque grand prix ne figure pas sur la page dédiée à ce dernier. En effet, il s'agit d'un lien vers la page du grand prix en général et non du grand prix d'une année précise.

Round ↕	Grand Prix ↕	Circuit ↕	Race date
1	Bahrain Grand Prix	 Bahrain International Circuit, Sakhir	28 March
2	Emilia Romagna Grand Prix	 Imola Circuit, Imola	18 April
3	Portuguese Grand Prix	 Algarve International Circuit, Portimão	2 May
4	Spanish Grand Prix	 Circuit de Barcelona-Catalunya, Montmeló	9 May
5	Monaco Grand Prix	 Circuit de Monaco, Monte Carlo	23 May

Figure 15 : Page wikipédia des saisons.

Finalement ce travail fastidieux est opérationnel de 1950 à 1999.

- Utilisation de l'API

La piste de l'API avait été oubliée car dans le schéma de la base de données proposé par cette dernière, aucune information concernant les grands prix n'est indiquée. Cependant, après des recherches approfondies dans la documentation de celle-ci, une table nommée '*aces*' fait référence aux grands prix, avec toutes les informations nécessaires. Le travail précédent inachevé et effectué en une dizaine d'heures a pu être fait de manière simple en seulement 5h, ce qui est un tournant.

Il s'agit de fichiers JSON, soit des dictionnaires imbriqués (voir annexe 14). Le travail a été de comprendre et de bien visualiser les différents dictionnaires ou listes afin de réussir à cibler uniquement les informations nécessaires. Également, nous récupérons le lien vers le grand prix d'une année. C'est l'information qu'il nous manquait depuis le début pour connaître le nombre de tours à effectuer pour chaque grand prix d'une saison.

Les problèmes rencontrés

Ce code est opérationnel de 1950 à 2025. En effet, les pages Wikipédia de certains grands prix de 2026, n'ayant pas encore eu lieu, n'existent pas. Ce code est d'autant plus robuste que s'il est relancé à la fin de la saison 2026, à condition que les pages web existent désormais et n'aient pas totalement été modifiées, les informations seront récupérées.

Le lien avec la table circuit

Une fois le travail de scraping réalisé (récupération et nettoyage des données brutes), il nous fallait faire le lien entre les Grands Prix et les circuits.

Le fait que les noms des circuits sont issus de la même source (API), nous ne rencontrons pas de problèmes de syntaxe ou d'orthographe lors du test d'égalité de ces derniers. Pour chaque grand prix, nous recherchons une correspondance entre la saison et le nom du circuit dans la table *Circuit*. Une première

tentative à été peu concluante. Le code était fonctionnel, mais beaucoup de circuits ne sont pas remplacés par leur identifiant. Cela était dû à une table *Circuit* incomplète. En effet, certains n'y figurent pas, ou bien les dates ou périodes ne concordent pas avec la réalité des faits (exemple le Circuit de Monaco utilisé en 1996 ne figure dans table que de 2015 à 2026).

Nous avons résolu ce problème en deux étapes :

- 1) D'une part une révision du scraping de la table *Circuit*, qui est devenu plus robuste.
- 2) D'autre part, le code permet l'identification de potentielles erreurs, malgré le fait que le nombre de ces dernières ait significativement diminué avec le nouveau fichier csv *Circuit* (nom fichier). Le code teste si le nom du circuit est repéré dans sa table et si les dates correspondent. Si les deux conditions sont respectées, alors l'identifiant remplace le nom initial. Sinon, une colonne *error* stipule si l'erreur est due à la non-présence du circuit ou si c'est seulement dû aux dates.

Finalement, nous retrouvons peu d'erreurs restantes, notamment une que nous avons corrigée directement à la main : le circuit de Las Vegas, initialement "Las Vega Street Circuit" dans la table *Circuit*, était nommé "Las Vegas Strip Street Circuit" dans la table *Grandprix*. Une correction au clavier directement au sein de la table *Circuit*, à permis d'éviter la majorité des erreurs, même si certaines demanderont un traitement manuel.

Critique des résultats

Nous avons choisi d'omettre l'heure de départ des grands prix, information que nous ne retrouvons sur aucun de nos sites.

f) Standings

Pour la table *Standings*, nous avons récupéré les informations suivantes :

- | | |
|--------------------------|----------------------------------|
| - Le grand prix; | - Sa position d'arrivée; |
| - Le pilote; | - L'incident éventuel de course; |
| - Sa position de départ; | - Les points gagnés; |
| | - Le nombre de tours complétés |

Nous récupérons au total un fichier CSV de près de 26 000 lignes.

Les différentes sources

Bien que des sites comme StatsF1 ou le site officiel de Formula 1 proposent des données complètes, leur extraction par scraping est nettement plus complexe et chronophage. Nous avons privilégié l'utilisation de l'API plutôt que le web scraping ce qui nous a permis d'être efficaces. La première étape essentielle est de bien analyser la structure du dictionnaire de l'API (voir annexe 15).

Les problèmes rencontrés

Nous n'avons pas eu de problème particulier dans la construction de la table, les données de l'API étant facilement récupérables. Ce qui a été fastidieux, c'est le temps que met python à récupérer les données. Nous sommes obligées de faire patienter le code 5 secondes minimum entre chaque récupération pour un seul grand prix. Il faut ensuite lancer le code plusieurs fois pour différents intervalles d'années. C'est ce qui a pris le plus de temps : vérifier que toutes les informations sont bien présentes, et, s'il en manque, relancer le code.

Nous avons récupéré les résultats de chaque pilote pour tous les grands prix ayant eu lieu depuis 1950.

Le lien avec les tables *Driver* et *Grand Prix*

Il faut joindre la table *Standings* avec les pilotes et les grands prix. Si la jointure a été faite facilement, nous avons été confrontés à une problématique d'harmonisation des noms de pilotes, similaire à celle rencontrée pour la table *Circuit*. Cette fois, comme le fichier contient près de 26 000 lignes, il est impossible de modifier à la main tous les noms des pilotes qui ne correspondent pas. A l'inverse, la table *Grandprix* n'a pas posé de problème pour le lien entre les tables, tous les grands prix sont recensés.

Afin d'identifier facilement les erreurs, nous avons créé une variable "error". Elle indique précisément l'état de la ligne : *NULL* si la jointure est complète, ou *pilote_introuvable* en cas d'échec de correspondance.

g) RaceDriver

Pour la table *RaceDriver*, nous avons collecté les informations suivantes pour chaque engagement à partir de 1974 :

- Le nom de l'écurie ;
- Le nom du pilote ;
- L'année de la saison ;
- Le numéro du pilote (spécifique à la saison et à l'écurie) ;
- Le manufacturier de pneus ;
- Le motoriste.

Au total, nous avons récupéré 2 182 lignes.

Les différentes sources

Contrairement aux étapes précédentes, aucune page Wikipédia ne fournit l'ensemble de ces informations de manière centralisée et uniforme. L'absence de pages statiques regroupant ces données compromet la réalisation de la table.

Nous avons rapidement constaté que les informations nécessaires ne sont accessibles que sur des sites web avec JavaScript. Cela nous a imposé de nous former à une nouvelle méthode d'extraction : Selenium.

- Page des numéros sur statsF1

Numéro 2

n	Pilote	Année	Nb
1	BARRICHELLO Rubens	2001 2002 2003 2004 2005	87
2	WEBBER Mark	2011 2012 2013	58
3	PROST Alain	1985 1989 1993	48
4	VANDOORNE Stoffel	2017 2018	41
5	COULTHARD David	1994 1999 2000	41
6	SARGEANT Logan	2023 2024	37
7	HAMILTON Lewis	2007 2010	36
8	BERGER Gerhard	1991 1992	32
9	MASS Jochen	1975 1977	31
10	REUTEMANN Carlos	1972 1979 1981	31
11	MANSELL Nigel	1990 1994	20

Figure 16 : Page du numéro 2 sur StatsF1..

Nous avons d'abord exploré les pages dédiées aux numéros de course (Figure 16). La page est faite telle que pour chaque numéro, nous accédons à la liste des pilotes ainsi qu'aux années durant lesquelles il a porté ce numéro. Bien que cela nous a permis d'associer un pilote à ses numéros au fil des ans, le lien avec les écuries reste complexe à établir. Cette étape fut toutefois cruciale pour la prise en main de l'outil Selenium, mais n'a pas été retenue.

- Page des pilotes engagés à chaque grand prix

Avec l'aide de notre référent, nous avons identifié une page plus exhaustive (Figure 17). Bien que son accès soit moins direct, elle regroupe par Grand Prix l'ensemble des données cibles : pilote, écurie, saison, numéro, moteur et pneumatiques.

N°	Pilote	Écurie	Châssis	Moteur	Pneu
2	Johnny CLAES	Ecurie Belge	Talbot Lago	T26C-DA Talbot	23CV L6 4.5 Englebert
4	Philippe ETANCELIN	Privé	Talbot Lago	T26C-DA Talbot	23CV L6 4.5 Dunlop
6	Yves GIRAUD-CABANTOUS	Privé	Talbot Lago	T26C Talbot	23CV L6 4.5 Dunlop
8	Louis ROSIER	Ecurie Rosier	Talbot Lago	T26C-DA Talbot	23CV L6 4.5 Dunlop
10	Henri LOUVEAU	Ecurie Rosier	Talbot Lago	T26C Talbot	23CV L6 4.5 Dunlop
12	George ABECASSIS	HW Motors	HWM	51 Alta	L4 c 1.5 Dunlop
14	Stirling MOSS	HW Motors	HWM	51 Alta	L4 c 1.5 Dunlop

Figure 17 : Page des pilotes engagés pour un Grand Prix.

La stratégie consiste à automatiser le parcours de chaque Grand Prix en utilisant le bouton de navigation « *suivant* » situé dans l'en-tête de la page. Ce traitement a démontré l'efficacité de Selenium : en environ 45 minutes, le script a parcouru l'historique complet pour générer un fichier de 30 000 lignes, incluant des doublons et des lignes vides qu'il a fallu traiter ultérieurement.

Problèmes rencontrés

Un obstacle majeur est apparu concernant la période antérieure à 1974. À cette époque, un pilote pouvait changer de numéro à chaque course, même au sein d'une écurie. Intégrer cette spécificité aurait nécessité d'inclure l'identifiant du Grand Prix dans la clé primaire de *RaceDriver*, entraînant un volume de données trop important (environ 20 000 lignes supplémentaires) et une redondance massive pour les saisons actuelles. De plus, cette période est également problématique à cause d'écuries « *privées* » créées par des pilotes autodidactes, rendant les associations impossibles avec ces dernières.

Nous avons donc pris des décisions pour pallier cela. Nous excluons les années de 1950 à 1973 afin préserver la lisibilité de la structure. Pour les écuries "privées" qui sont peu nombreuses au vu de la période, leur nom est remplacé par "NULL" lors de la jointure.

Critique des résultats

L'ajout du motoriste et du manufacturier de pneus dans cette table provoque une redondance (les pilotes d'une même équipe partageant ces éléments), mais c'est le seul moyen de garantir la traçabilité de ces informations qui varient chaque saison sans ajouter une table.

Les liens et le nettoyage

Le lien de la table *Racedriver* avec les autres tables a nécessité des traitements spécifiques :

Table *Season* : Le lien est direct via l'année de la saison.

Table *Driver* : Pour pallier les écarts d'orthographe et de syntaxe, nous avons implémenté un test de correspondance flexible. Le script vérifie la présence du nom et du prénom dans la chaîne de caractères brute récupérée (exemple : vérifier « ryo » et « hirakawa » dans « ryohirakawa* » préalablement nettoyée : mise en minuscule et sans espace).

Table *Team* : Nous avons constaté que les dénominations de la colonne « écurie » ne correspondent pas toujours à celles de notre table *Team*. En approfondissant l'analyse, il s'est avéré que le nom de l'écurie utilisé sur le site coïncide souvent avec le nom du châssis répertorié dans nos travaux précédents (faisant écho aux pages scrapées pour la table *Team*). Nous avons donc pris une décision : dès que le nom du châssis est identifiable au sein de l'appellation fournie dans la colonne écurie, nous le substituons par ce dernier, ce qui a permis de valider le lien pour la grande majorité des écuries.

Enfin, pour assurer la transparence du processus malgré les délais impartis, nous avons créé une variable "error". Elle indique précisément l'état de la ligne : *NULL* si tous les liens sont trouvés, ou mentionne *TEAM*, *DRIVER* ou *TEAM & DRIVER* en cas d'échec de correspondance.

Tous les scripts pythons ainsi que les fichiers csv sont disponibles dans l'annexe technique (fichier zip).

4) Les problèmes communs rencontrés

a) Wikipédia

Le recours à Wikipédia a révélé une instabilité structurelle importante, obligeant à un nettoyage approfondi et à une adaptation constante du code de scraping. Voici les principales difficultés rencontrées :

Hétérogénéité du formalisme et de la syntaxe

L'absence de standardisation stricte entre les pages a nécessité des ajustements :

- Variabilité des intitulés : Certains champs changent de nom selon l'époque (ex: le champ «Nombre de manches» devient «Épreuves» dès 2011).
- Pollution textuelle : Les données numériques sont souvent polluées par du texte («participants», «équipes») ou des annotations Wikipédia (ex: [1], [b]), nécessitant un nettoyage par expressions régulières.
- Unités de mesure : Les longueurs de circuits ne sont pas uniformes, alternant parfois entre miles et kilomètres.

Complexité de la structure HTML

L'extraction ciblée des informations a représenté l'un des plus grands défis techniques :

- Identification des balises : La structure des fichiers HTML varie d'une page à l'autre. Il a fallu analyser finement l'arborescence pour isoler les bonnes balises dans un flux de code très répétitif.
- Emplacement variable : La position du tableau cible dans la page n'est pas fixe (il peut s'agir du premier ou du deuxième tableau selon la saison par exemple), rendant l'indexation automatique complexe.
- Lignes parasites : La présence de lignes de bas de tableau (ex: "Sources") génère des erreurs lors de l'intégration des données.

Anomalies de continuité des données

Certaines erreurs touchent directement à la logique des calendriers de Formule 1 :

- Cellules vides par répétition : Lorsqu'un circuit est utilisé pour deux Grands Prix consécutifs, l'information est parfois omise sur la deuxième ligne du code HTML, créant des ruptures dans les données.
- Incompatibilité des périodes : Les formats de dates et de périodes de service des circuits ne sont pas directement exploitables pour des jointures SQL, imposant un reformatage complet.
- Données faussées : Nous avons constaté à plusieurs reprises que les données fournies par le site étaient fausses, ne délivrant pas des informations vérifiées pour notre base de données.

Stratégie de contournement

Pour pallier ces instabilités tout en respectant les limites de requêtes imposées par Wikipédia, nous avons opté pour un traitement fragmenté (table par table). L'ajout d'une variable de suivi d'erreur a permis de traiter le problème pas à pas et de garantir la robustesse finale du fichier CSV, malgré le caractère non homogène de la source. Malgré cela, il reste à noter que les informations des CSV sont potentiellement faussées étant donné le statut non vérifié des informations fournies.

b) L' API Ergast

Malgré le fait que l'utilisation de l'API soit une solution conséquente pour gagner du temps, son exploitation présente deux obstacles principaux.

D'une part, la fiabilité des données entraîne une certaine réserve : l'API puisant ses informations directement dans Wikipédia, elle hérite des potentielles erreurs factuelles ou approximations de cette source collaborative. D'autre part, nous avons dû composer avec une limite de requêtes stricte imposée par le serveur. Cette contrainte technique a nécessité l'implémentation de fonctions `time.sleep()` dans notre code, allongeant considérablement le temps d'exécution. La gestion de ces pauses forcées a rendu le processus de récupération particulièrement fastidieux, nous obligeant à surveiller étroitement le script pour éviter toute interruption ou perte de données.

5) Les finitions à apporter

Malgré un travail conséquent mené sur les douzes semaines de projets, nous n'atteignons pas l'idéal que nous nous étions fixé.

Le contenu des tables

Certaines tables sont bel et bien terminées, à l'image des tables *Driver*, *Circuits* ou *Team*. Cependant, les tables nécessitant des associations complexes (*Grandprix*, *Racedriver*, *Standings*, *Season*) requièrent encore des ajustements. Des erreurs mineures laissent certains liens en suspens et demandent un traitement manuel, notamment pour vérifier l'orthographe de certains noms, la concordance des dates pour les circuits ou encore l'uniformisation des noms d'équipes. De plus, le temps nous a manqué pour finaliser certains liens, comme pour la table *Season*, ainsi que pour l'implémentation complète de la table *TestDriver*.

Le format de la base de données

Certains attributs encore présents sont voués à disparaître et ne seront pas intégrés à la base de données finale. C'est le cas de *seFixedNb*, qui n'a plus d'intérêt au vu de la réalité historique de la Formule 1. De plus, le format de certains attributs aurait dû être revu : l'attribut « *Dates* » de la table *Circuits*, qui représente une période, gagnerait à être décomposé en deux colonnes, *cstart* et *cend*, de manière analogue à la structure de la table *Team*.

Enfin, nous espérons apprendre à transférer nos fichiers CSV vers une base de données réelle, une étape que nous n'avons malheureusement pas pu concrétiser dans les délais impartis.

III - Prise de recul et perspectives

Avec le recul, notre dépendance à Wikipédia a constitué le principal frein à l'efficacité du projet. Bien que ce site soit une mine d'informations, son instabilité structurelle et son caractère collaboratif ont nécessité un travail de nettoyage disproportionné.

Si nous devions recommencer ce projet, nous changerions de stratégie :

- **Priorité au dynamique** : Nous délaisserions Wikipédia pour nous concentrer dès le départ sur des sources plus robustes et officielles comme le site de la FIA ou StatsF1, même si l'utilisation de JavaScript les rends plus difficiles à scraper.
- **Maîtrise de Sélénium** : Plutôt que de choisir la "facilité" du statique, nous aurions investi plus de temps dès le début dans une formation approfondie sur Sélénium. Cette méthode, qui nous paraissait impressionnante au départ, s'est révélée tout à fait abordable et bien plus puissante pour atteindre des données de qualité.
- **Sources fiables** : Nous effectuerions des recherches plus poussées sur la fiabilité des sites avant de coder, afin d'éviter de passer des heures sur des sources qui s'avèrent finalement incomplètes ou fausses.

Un autre point d'amélioration concerne l'anticipation des liens entre tables. Dans notre rétroplanning, nous avons placé la vérification des correspondances (noms de pilotes, circuits) en fin de parcours. Cette organisation a provoqué beaucoup de difficultés : découvrir les erreurs de syntaxe au moment de lier les tables nous a obligées à reprendre des codes déjà finalisés, ce qui est très chronophage. À l'avenir, nous intégrerions des tests de cohérence dès la phase de scraping ou nous prioriserions des sources identiques, ce qui pourrait privilégier des orthographes uniformes. Cela permettrait de traiter les anomalies (comme les pilotes corrigés à la main) au fur et à mesure, évitant ainsi un traitement manuel de dernière minute sous la pression du temps.

Conclusion

Nous tirons un bilan très positif de notre organisation globale. Le rythme de nos réunions a été le moteur du projet, nous imposant une cadence soutenue et un suivi rigoureux. Bien que le volume de travail réel (environ 50h par personne, en moyenne) ait largement dépassé nos estimations initiales (environ 37h par personne), nous avons su adapter notre planning et nos priorités pour aboutir à des fichiers CSV cohérents.

Techniquement, ce projet a été un véritable parcours d'apprentissage. Si les bibliothèques BeautifulSoup4 et Requests ont été nos outils privilégiés pour les sites sans Javascript comme Wikipédia, nous avons dû faire preuve d'adaptation face à l'instabilité structurelle de ces sources. La découverte et l'auto-formation sur Selenium ont été cruciales pour accéder aux données des sites avec Javascript, palliant ainsi les limites du scraping classique. Nous avons également appris à composer avec les contraintes des API et la nécessité d'un nettoyage rigoureux des données.

Malgré l'absence de la table *Testdriver* et le fait que nous n'ayons pas pu finaliser l'intégration complète en base de données SQL par manque de temps, les résultats obtenus constituent une ressource pédagogique solide.

La réactivité de notre référent a également été un facteur clé de succès, nous permettant de débloquent des situations techniques complexes rapidement. Pour finir, choisir un sujet qui nous passionnait a rendu ce projet très formateur. Cela nous a permis de voir le fossé qu'il peut y avoir entre la théorie des bases de données et la réalité du terrain. C'est une expérience qui nous sera vraiment utile pour nos futurs projets.

Sitographie et Bibliographie

- *beautifulsoup4: Screen-scraping library*. Python. *PyPI*,
<https://www.crummy.com/software/BeautifulSoup/bs4/>.
- « Best 7 Python Web Scraping Libraries in 2025 ». *ScraperAPI*,
[https://www.scraperapi.com/web-scraping/python/libraries/..](https://www.scraperapi.com/web-scraping/python/libraries/)
- *Category:Formula One cars by car number* — *Wikimedia Commons*.
https://commons.wikimedia.org/wiki/Category:Formula_One_cars_by_car_number.
Consulté le 9 avril 2026.
- « Championnat du monde de Formule 1 1951 ». *Wikipédia*, 24 septembre 2025.
Wikipedia,
https://fr.Wikipédia.org/w/index.php?title=Championnat_du_monde_de_Formule_1_1951&oldid=229236324.
- Collart, Thomas. *Comment créer un fichier CSV en python ?*
<https://www.commentcoder.com/python-fichier-csv/>.
- *Comment faire du web scraping avec Selenium en Python?*
<https://www.pythoniaformation.com/blog/tutoriels-python-par-categories/automatiser-avec-python/comment-faire-web-scraping-selenium-python>.
- *Constructeurs • STATS F1*. <https://www.statsf1.com/fr/constructeurs.aspx>.
- « Csv — CSV File Reading and Writing ». *Python Documentation*,
<https://docs.python.org/3/library/csv.html>.
- *datetime — Types de base pour la date et l'heure — Documentation Python 3.10.20*.
<https://docs.python.org/fr/3.10/library/datetime.html>.
- « Écurie de Formule 1 ». *Wikipédia*, 16 février 2026. *Wikipedia*,
https://fr.Wikipédia.org/w/index.php?title=%C3%89curie_de_Formule_1&oldid=233354452.

- « Formule 1 ». *Wikipédia*, 2 avril 2026. *Wikipedia*,
https://fr.Wikipédia.org/w/index.php?title=Formule_1&oldid=234734117.
- « Formule 1 : actualité, info, vidéo et résultats de tous les grands prix de F1 ». *L'Équipe*, <https://www.lequipe.fr/Formule-1/>.
- *Grande-Bretagne 1950 - Engagés • STATS F1*.
<https://www.statsf1.com/fr/1950/grande-bretagne/engages.aspx>.
- « Les 7 meilleures bibliothèques Python pour le Web Scraping ». *Bright Data*,
<https://brightdata.fr/blog/donnees-web/python-web-scraping-libraries>.
- « List of Formula One Circuits ». *Wikipedia*, 5 avril 2026. *Wikipedia*,
https://en.Wikipédia.org/w/index.php?title=List_of_Formula_One_circuits&oldid=1347306675.
- « List of Formula One Grands Prix ». *Wikipedia*, 7 avril 2026. *Wikipedia*,
https://en.Wikipédia.org/w/index.php?title=List_of_Formula_One_Grands_Prix&oldid=1347611185.
- « Liste des femmes pilotes de Formule 1 ». *Wikipédia*, 26 février 2026. *Wikipedia*,
https://fr.Wikipédia.org/w/index.php?title=Liste_des_femmes_pilotes_de_Formule_1&oldid=233648617.
- « Liste des pilotes de Formule 1 ». *Wikipédia*, 8 mars 2026. *Wikipedia*,
https://fr.Wikipédia.org/w/index.php?title=Liste_des_pilotes_de_Formule_1&oldid=233908977.
- « McLaren Racing ». *Wikipédia*, 19 mars 2026. *Wikipedia*,
https://fr.Wikipédia.org/w/index.php?title=McLaren_Racing&oldid=234225450.
- *Mocodo online*. <https://www.mocodo.net/>.
- « Numéro permanent en Formule 1 ». *Wikipédia*, 17 décembre 2025. *Wikipedia*,
https://fr.Wikipédia.org/w/index.php?title=Num%C3%A9ro_permanent_en_Formule_1&oldid=231563363.
- *Outils de scraping: Notre classement en 2025*.
<https://www.ecole.cube.fr/blog/meilleurs-outils-scraping..>

- Perry, Jess. *jolpica/jolpica-f1*. 1 octobre 2023, Python.. *GitHub*,
<https://github.com/jolpica/jolpica-f1>.
- « Re — Regular Expression Operations ». *Python Documentation*,
<https://docs.python.org/3/library/re.html>.
- *Requests: HTTP for Humans™ — Requests 2.33.1 documentation*.
<https://requests.readthedocs.io/en/latest/>.
- « Scrapy ». *Scrapy*, <https://scrapy.org>.
- *Statistiques Numéro - Numéro - Pilotes • STATS F1*.
<https://www.statsf1.com/fr/statistiques/numero/pilote/pilotes.aspx>.
- *STATS F1*. <https://www.statsf1.com/fr/default.aspx>.
- Ughetto, Laurent. « Présentation de la base de données ».
- Ughetto Laurent. « Web scraping pour enrichir une base de données ».

Annexe

Annexe 1:

Réunion du 11/03/26 - S8

- Rajout des pilotes femmes : manque de fiabilité de Wikipédia car manque d'informations
- Table pilote : problème manque des dates, avec information en trop ou deux nationalités différentes => à vérifier
- Table team : problème car on retrouve des période d'activité et d'inactivité. On a pas assez d'information avec wikipedia. Pour l'instant, garder les informations communes en précisant qu'elles ne valent pas forcément pour les périodes antérieurs
 - La nationalité : garder celle du premier tableau
 - Si c'est homogène sur wikipedia, rajouter la localisation
- Pour le rendu :
 - Fichier CSV : garder les fichiers initiaux et rendre des fichiers "propres", faire les associations
 - Scripts python : garder les commentaires utiles pour comprendre le code

Annexe 2 :

TER L3 MIASHS 2025 - 2026

Sujet - Web scraping pour enrichir une base de données

Rapport initial pour le 19/01/2026

*Rédigé par BERNARD Emmy, EON Anaëlle, BADOUARD Youna-Marie,
ORZAKIEWICZ Juliette*

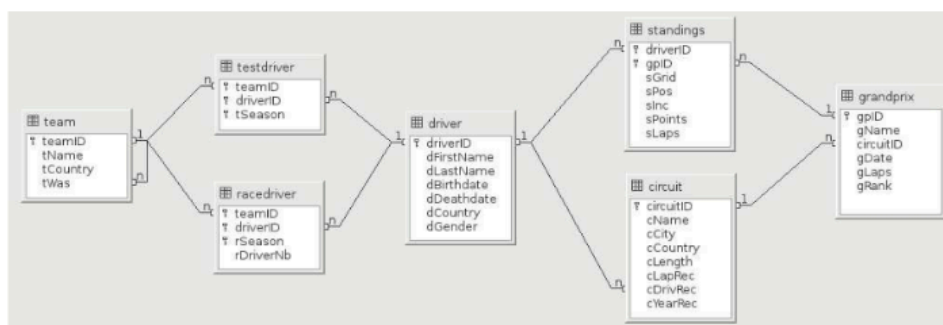
Encadré par Laurent Ughetto

1. Introduction

Ce rapport s'inscrit dans le cadre de notre projet de recherche encadré portant sur le web scraping appliqué à l'enrichissement d'une base de données. Il est destiné à introduire notre sujet, l'expliquer et présenter notre planning d'organisation, ainsi que le déroulement de nos deux premières semaines de travail. Nous allons d'abord faire une restitution du sujet afin d'en expliquer le contexte et de présenter nos objectifs. Puis, nous aborderons la planification de notre travail sur les douze semaines prévues pour ce projet et, enfin, comment se sont déroulées les deux premières semaines par rapport à nos attentes.

2. Restitution du sujet

Notre sujet concerne une base de données sur la Formule 1 déjà initialisée en amont. Elle a été créée dans le but d'aider à l'apprentissage du langage d'interrogation SQL. Elle est composée des 7 tables suivantes :



La base de données contient des informations concernant les écuries, les pilotes, les résultats de course, les circuits et les Grands Prix.

Cette dernière parcourt seulement la période de janvier 2013 à septembre 2014, ce qui représente une saison et demie de Formule 1. La base de données comporte également les informations de tous les pilotes champions du monde depuis 1950. Les tables ne comptabilisent donc pas les données de toutes les saisons de ce sport : il manque certains pilotes, certaines écuries, etc.

Notre objectif est donc d'enrichir cette base de données à l'aide du web-scraping, en rédigeant un programme capable de récupérer les données sur la Formule 1 de 1950 à 2025 à partir de pages web. La finalité de ce travail est d'alimenter la base de données initiale incomplète pour un cours de langage SQL en Master 1 Mathématiques Appliquées et Statistique.

Il est nécessaire d'identifier des sites web fiables à propos de la Formule 1, d'extraire les données grâce aux méthodes de Web Scraping et éventuellement d'utiliser des API afin de simplifier le codage.

3. Planification prévisionnelle

Afin de commencer la répartition de notre travail, nous avons organisé une première réunion le mardi 13/01/2026, toutes les 4, la veille de la réunion avec notre professeur référent : le "client". Nous avons par la suite choisi de conserver cette réunion hebdomadaire.

Lors de notre première réunion, nous avons tout d'abord défini notre planning, ci-dessous :

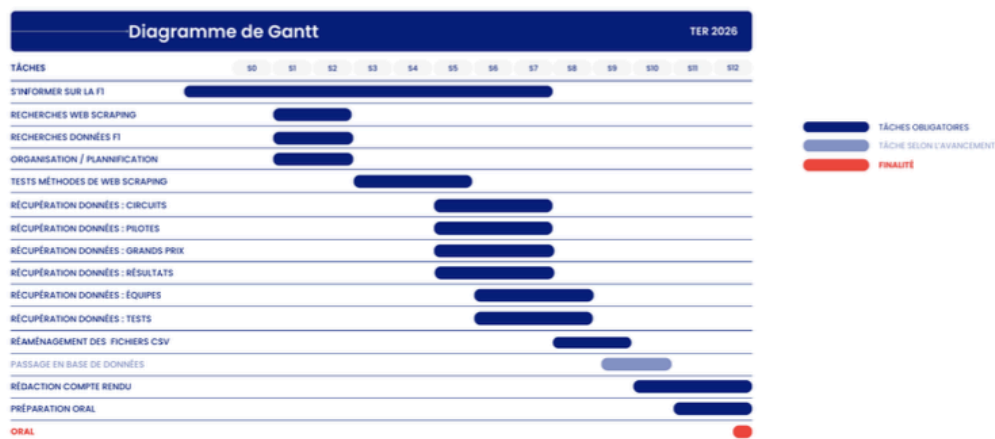
	ÉTAPES :	SEMAINE	DURÉE ESTIMÉE	RÉPARTITION	QUI ?
1	S'informer sur la F1	vacances + jusqu'à S7	personnel	Individuelle	
2	Se documenter sur le web scraping - trouver différentes méthodes	S1-S2	3h par personne	Travail pour 2	EMMY ANAËLLE
3	Trouver les informations nécessaires sur la F1	S1-S2	3h par personne	Travail pour 2	YOUNA-MARIE JULIETTE
4	S'organiser, planifier	S1-S2	3h par personne	Une fois les étapes validées, une tâche chacune	Restitution sujet : JULIETTE Planification : EMMY Répartition du travail : ANAËLLE Bibliographie : YOUNA-MARIE
5	Tests des méthodes de web scraping	S3-S5	7h par personne	Selon le nombre de méthodes trouvées : une ou deux par méthode. Faire un petit compte rendu des tests effectués sur des données de F1. => Choisir les méthodes + justifications	Selon le nombre de méthode testé, se mettre en binôme.
6	Codage : Récupération des différentes informations (Circuits, pilotes, ...)	S5-S7	12h par personne	Répartition en fonction des thèmes.	YOUNA-MARIE JULIETTE ANAËLLE EMMY
7	Réaménagement des fichiers csv pour avoir une forme proche des tableaux demandés et mise en place des liens entre ces fichiers	S8-S9	6h par personne	Mutualisation de nos fichiers csv obtenus à l'étape suivante afin d'obtenir une structure qui se rapproche des tables.	2 formats de tables chacune : YOUNA-MARIE JULIETTE ANAËLLE EMMY
8	Passer en base de données	S9-S10 si on a le temps	?	Voir selon l'avancement.	?
9	Rédaction du compte rendu	S10-S11-S12 à faire en réalité tout au long du projet	6h par personne	Établir le plan et répartir les parties. Prévoir des relectures communes.	YOUNA-MARIE JULIETTE ANAËLLE EMMY
10	Préparation Oral	S10-S11-S12	6h par personne	(diapo, entraînements) répartition,	YOUNA-MARIE JULIETTE ANAËLLE EMMY

Cette méthode nous a semblé être la meilleure solution pour commencer notre travail, car nous ne savions pas réellement par où débiter ni comment organiser nos idées. Ce tableau nous a permis de structurer le projet et d'avoir une vision globale des tâches à accomplir.

Nous avons, pour commencer, réfléchi ensemble aux différentes "étapes" du projet, mais nous avons rencontré quelques difficultés pour les extraire clairement au début. C'est lors de notre première réunion encadrée par le professeur que nous sommes parvenus à dégager les différentes étapes.

Pour la colonne "semaine", l'intelligence artificielle nous a aidées car nous n'avions pas vraiment d'idée de combien de temps les tâches allaient nous prendre. Ce qu'elle nous a dit nous a paru assez cohérent.

Cela nous a permis de situer chaque étape dans le temps, de voir quelles étapes allaient se chevaucher etc. Nous avons pu réaliser un diagramme de Gantt :



Nous avons également prévu, grâce à la colonne "durée estimée", le temps nécessaire, en heures, à la réalisation de chaque tâche, afin de mieux répartir notre charge de travail et d'éviter le retard. Nous avons retenu une base de 4 heures hebdomadaires par personne, en déduisant au préalable environ 1 heure de réunion chacune.

Pour finir avec le tableau, nous avons inclus les colonnes "répartition" et "qui" afin de nous distribuer clairement le travail. Chaque personne sait donc ce qu'elle a à faire pour chaque semaine, cela nous a donc permis de définir clairement l'organisation du groupe ainsi que de responsabiliser chacune d'entre nous et de travailler de manière plus efficace et organisée.

Afin d'optimiser notre travail, nous avons également mis en place un document type qui fait office de "journal de bord" :

Étape :	
Nom - Prénom	
Durée estimée :	
Durée réelle :	
Outils :	
Liens :	

Le principe est qu'à chaque étape, chacune le remplisse de manière régulière en y notant la durée que nous avons estimée et la durée réelle que cela lui a pris, ainsi que les outils

utilisés et les liens. Si la durée réelle ne correspond pas à la durée estimée, alors nous expliquons pourquoi en quelques lignes. Il permet de garder une trace de nos différentes tâches.

Pour finir, nous avons également mis en place un espace-cours cursus que nous avons agencé. Cela nous permet de mettre en ligne tous nos documents et toutes nos sources ainsi que de nous organiser par semaine et de noter les points clés des réunions. Cursus nous permet aussi de communiquer entre nous 5 (avec le professeur référent) assez facilement grâce au forum de discussion.

C'est une alternative à l'application Trello, que nous n'avons pas trouvée très facile d'emploi. Voici un petit aperçu de l'espace-cours :



4. Déroulement des premières semaines

Durant les deux premières semaines, nous n'avons pas seulement travaillé sur la planification. En effet, une partie importante de recherche devait être effectuée rapidement pour pouvoir passer à la partie centrale de notre projet : le web scraping.

Pour ce faire, nous avons recherché les sites utiles, dans lesquels nous retrouvions toutes les informations à propos de la Formule 1. La difficulté était de retrouver toutes les données à scraper en limitant le nombre de sources. Sur une dizaine de sites initialement, nous retenons alors Wikipédia : pour son format fixé et StatsF1 : un site amateur mais très complet. Enfin, nous avons trouvé une API qui nous permettrait également de récupérer un grand nombre de données facilement.

En amont, deux d'entre nous se sont renseignés sur les méthodes de web scraping. Nous sommes alors passées par des sites amateurs, des forums ou des classements de méthodes afin de découvrir, comparer et faire un premier tri dans ce qui existe. Certaines pages étaient exclusivement en anglais, ce qui freinait notre compréhension. Nous avons alors présélectionné, avant les tests concrets, 4 bibliothèques Python : Selenium, Requests, BeautifulSoup4 et Scrapy.

Nous avons donc atteint nos objectifs pour ces 2 premières semaines et nous commencerons les tests sur des données de la F1, dès la semaine 3.

Bibliographie :

« Comment faire du web scraping avec Selenium en Python? » Consulté le 13 janvier 2026. <https://www.pythoniaformation.com/blog/tutoriels-python-par-categories/automatiser-avec-python/comment-faire-web-scraping-selenium-python>.

Fáwólé, John. « Best 7 Python Web Scraping Libraries in 2025 ». ScraperAPI, 26 janvier 2025. <https://www.scraperapi.com/web-scraping/python/libraries/>.

GitHub. « Jolpica-F1/Docs/README.Md at Main · Jolpica/Jolpica-F1 ». Consulté le 13 janvier 2026. <https://github.com/jolpica/jolpica-f1/blob/main/docs/README.md>.

Groc, Thomas. « Outils de scraping: Notre classement en 2025 ». 11 juillet 2025. <https://www.ecole.cube.fr/blog/meilleurs-outils-scraping>.

L'Équipe. « Formule 1 : actualité, info, vidéo et résultats de tous les grands prix de F1 ». Consulté le 13 janvier 2026. <https://www.lequipe.fr/Formule-1/>.

Quelles sont les meilleures librairies Python actuelles pour le Web Scraping et pourquoi? . s. d. https://www.reddit.com/r/webscraping/comments/1i0hthe/what_are_the_current_best_python_libs_for_web/?tl=fr.

« Requests: HTTP for Humans™ — Requests 2.32.5 documentation ». Consulté le 13 janvier 2026. <https://requests.readthedocs.io/en/latest/>.

Richardson, Leonard. *beautifulsoup4: Screen-scraping library*. Python. Released 30 novembre 2025. <https://www.crummy.com/software/BeautifulSoup/bs4/>.

Scrapy. « Scrapy ». Consulté le 13 janvier 2026. <https://scrapy.org>.

« STATS F1 ». Consulté le 15 janvier 2026. <https://www.statsf1.com/fr/default.aspx>.

Ughetto, Laurent. « Présentation de la base de données « Formula One ». 09/23.

Ughetto, Laurent. « Web scraping pour enrichir une base de données ». s. d.

Wikipédia. « Liste des pilotes de Formule 1 ». Consulté le 15 janvier 2026. https://fr.wikipedia.org/w/index.php?title=Liste_des_pilotes_de_Formule_1&oldid=232436010.

Zanini, Antonello. « Les 7 meilleures bibliothèques Python pour le Web Scraping ». Bright Data. Consulté le 15 janvier 2026. <https://brightdata.fr/blog/donnees-web/python-web-scraping-libraries>.

SciencesPO. « Citer ses sources et rédiger une bibliographie ». Juillet 2024.

Annexe 3 :

	ÉTAPES :	SEMAINE	DURÉE ESTIMÉE	DURÉE RÉELLE	RÉPARTITION	QUI ?
1	S'informer sur la F1	vacances	personnel		chacune de son côté	
2	Se documenter sur le web scraping - trouver différentes méthodes	S1-S2	3h par personne	Emmy : 2h Anaëlle : 3h	Recherches chacune de notre côté puis mise en commun pour en garder 3	EMMY ANAËLLE
3	Trouver les informations nécessaires sur la F1	S1-S2	3h par personne	Youna-Marie : 3h Juliette : 3h	2 sur les sites de F1.	YOUNA-MARIE JULIETTE
4	S'organiser, planifier	S1-S2	3h par personne	Emmy : 5h Anaëlle : 2h Youna-Marie : 1h Juliette : 2h	JULIETTE : Restitution sujet EMMY : Planification ANAËLLE : Répartition du travail YOUNA-MARIE: Bibliographie	JULIETTE EMMY ANAËLLE YOUNA-MARIE
5	Tests des méthodes de web scraping	S3-S4	7h par personne	Emmy : 8h45 Anaëlle : 8h Youna-Marie : 8h Juliette : 6h	JULIETTE : Selenium EMMY : Scrapy ANAËLLE & YOUNA-MARIE : Requests & Bs4	JULIETTE EMMY ANAËLLE YOUNA-MARIE
6	Codage	S5-S10 +semaine des vacances	12h par personne	EMMY : Table Grand Prix ⇒ 15h Table season ⇒ 8h30 Table Race Driver ⇒ 14h45 Anaëlle : 22h Youna-Marie : ⇒ table pilote : 8h ⇒ table team : 20h Juliette: ⇒ table circuits: 15h ⇒ table standings : 8h	Répartition des différentes tables selon les avancements de chacunes	YOUNA-MARIE JULIETTE ANAËLLE EMMY
7	Associations des tables	S8-S10	6h par personne	Anaëlle : NULL Youna-Marie : NULL	mutualisation de nos fichiers csv	Concerne les tables Circuit, Standings,

	(relier les numéros des identifiants)			Juliette : 3h Emmy : 3h	obtenus à l'étape suivante.	Season, Grandprix et Racedriver
8	Passer en base de données	S9-S10		MANQUE DE TEMPS		?
9	Rédaction du compte rendu	S10-S12	6h par personne	Anaëlle : 22h Emmy : 26h Youna-Marie : 22h Juliette : 18h		YOUNA-MARIE JULIETTE ANAËLLE EMMY
10	Préparation Oral	S12 et semaine de vacances	6h par personne	Anaëlle : 8h Emmy : 8h Youna-Marie : 8h Juliette : 8h	(diapo, répartition, entraînements)	YOUNA-MARIE JULIETTE ANAËLLE EMMY

Annexe 4 :

	ÉTAPES :	SEMAINE	DURÉE ESTIMÉE	RÉPARTITION	QUI ?
1	S'informer sur la F1	vacances + jusqu'à S7	personnel	Individuelle	
2	Se documenter sur le web scraping - trouver différentes méthodes	S1-S2	3h par personne	Travail pour 2	EMMY ANAËLLE
3	Trouver les informations nécessaires sur la F1	S1-S2	3h par personne	Travail pour 2	YOUNA-MARIE JULIETTE
4	S'organiser, planifier	S1-S2	3h par personne	Une fois les étapes validées, une tâche chacune	Restitution sujet : JULIETTE Planification : EMMY Répartition du travail : ANAËLLE Bibliographie : YOUNA-MARIE
5	Tests des méthodes de web scraping	S3-S5	7h par personne	Selon le nombre de méthodes trouvées : une ou deux par méthode. Faire un petit compte rendu des tests effectués sur des données de F1. => Choisir les méthodes + justifications	Selon le nombre de méthode testé, se mettre en binôme.
6	Codage : Récupération des différentes informations (Circuits, pilotes, ...)	S5-S7	12h par personne	Répartition en fonction des thèmes.	YOUNA-MARIE JULIETTE ANAËLLE EMMY
7	Réaménagement des fichiers csv pour avoir une forme proche des tableaux demandés et mise en place des liens entre ces fichiers	S8-S9	6h par personne	Mutualisation de nos fichiers csv obtenus à l'étape suivante afin d'obtenir une structure qui se rapproche des tables.	2 formats de tables chacune : YOUNA-MARIE JULIETTE ANAËLLE EMMY
8	Passer en base de données	S9-S10 si on a le temps	?	Voir selon l'avancement.	?
9	Rédaction du compte rendu	S10-S11-S12 à faire en réalité tout au long du projet	6h par personne	Établir le plan et répartir les parties. Prévoir des relectures communes.	YOUNA-MARIE JULIETTE ANAËLLE EMMY
10	Préparation Oral	S10-S11-S12	6h par personne	(diapo, répartition, entraînements)	YOUNA-MARIE JULIETTE ANAËLLE EMMY

Annexe 5 :

Fiche récapitulative qui se base sur un fichier de base récupéré sur : <https://github.com/ABD-Z/Scrapy-startup-project.git>

1. Installer Scrapy avec : "pip install scrapy" dans le terminal.
2. Créer un projet : "scrapy startproject [NomDuProjet]" ou entrer directement dans le projet : "cd [NomDuProjet]". Cela crée l'arborescence suivante :
3. Remplacer dans `spiders/quotes.py` les attributs de classe `allowed_domains` et `start_urls` par les URLs que l'on souhaite scraper

```
▼ scrapy_startup_spider
  > __pycache__
  ▼ spiders
    > __pycache__
    • __init__.py
    • quotes.py
    • __init__.py
    • items.py
    • middlewares.py
    • pipelines.py
    • settings.py
```

```
class QuoteSpider(scrapy.Spider):
    name = 'quotes'
    allowed_domains = ['fr.wikipedia.org']
    start_urls = ['https://fr.wikipedia.org/wiki/Liste_des_pilotes_de_Formule_1']
```

4. Avant de commencer à scraper, il faut identifier les balises utiles. Alors dans le terminal, on lance une première commande : "scrapy shell".
5. Ensuite, on lance `fetch('url_a_scraper')`

```
In [1]: fetch("https://fr.wikipedia.org/wiki/Liste_des_pilotes_de_Formule_1")
2026-01-28 21:05:16 [asyncio] DEBUG: Using selector: KqueueSelector
2026-01-28 21:05:16 [scrapy.core.engine] INFO: Spider opened
2026-01-28 21:05:17 [scrapy.core.engine] DEBUG: Crawled (200) <GET https://fr.wikipedia.org/robots>
> (referer: None)
2026-01-28 21:05:18 [scrapy.core.engine] DEBUG: Crawled (200) <GET https://fr.wikipedia.org/wiki/L
des_pilotes_de_Formule_1> (referer: None)
```

6. Cette commande renvoie un objet "response". Il faut, à ce moment, trouver dans le code source de la page web les sélecteurs qui permettent d'attraper les informations souhaitées. Par exemple ici, la liste des noms de pilotes.

```
<hr>
▶ <div class="liste-horizontale" style="text-align:center; font-size:89%;">...</div> = $0
▼ <div class="mw-heading mw-heading3">
  <h3 id="A">A</h3>
  ▶ <span class="mw-editsection">...</span>
  </div>
  ▼ <ul>
    ▼ <li>
      ▶ <span class="flagicon" data-sort-value">...</span>
      <a href="/wiki/George_Abecassis" title="George Abecassis">George Abecassis</a>
      " - 2 GP (1951-1952)"
```

On quitte la fonction `shell` de scrapy : "quit"

7. Après avoir trouvé et vérifié ces premiers sélecteurs, on modifie la fonction `parse` qui permet de découper, d'analyser une structure de données.

```
def parse(self, response):
    for quote_bloc in response.css('.mw-parser-output > ul > li'):
        pilote = quote_bloc.css('>a::text').get()
        yield QuoteItem(Nom=pilote)
```


8. Il faut aussi modifier le script nommé "*items.py*" qui était pré-rempli pour le tutoriel sur lequel nous nous appuyons. On définit les éléments que l'on veut récupérer, pour l'instant les noms.

```
class QuoteItem(scrapy.Item):  
    # define the fields for your item here like:  
    scrapy.Field()
```

9. On tente alors un premier test. Pour exécuter scrapy et récupérer un fichier on lance cette commande (avec nom et format du fichier au choix) :

```
in % scrapy crawl quotes -o fichier.json  
d (bot: scrapy_startup_spider)
```

Un nouveau document est créé dans l'arborescence.

Conclusion :

- Prise en main compliquée
- Nécessite un tutoriel en ligne avec un fichier préconstruit pour l'exemple et donc pas forcément adapté.
- Pas de réussite concrète lors des tests.

Annexe 6 :

Les importations à faire :

```
import requests
from bs4 import BeautifulSoup
```

- **Requests** sert à **recupérer** le contenu (le code HTML brut).
- **BeautifulSoup (BS4)** sert à **analyser** (parser) et extraire les données de ce code.

On marque ça au début de chaque code :

```
url = "https://fr.Wikipédia.org/wiki/Charles_Leclerc_(pilote_automobile)"
headers = {"User-Agent": "Mozilla/5.0"}
response = requests.get(url, headers=headers)
```

Petite astuce : Pour éviter de travailler dans le vide, on vérifie toujours que ça marche avec cette commande :

```
if response.status_code == 200: # vérifier que ça marche.
```

status = response.status_code → Si 200 alors ok, si 404 alors not found, donc on travaille dans le vide.

Initialisation de BS4 :

```
soup = BeautifulSoup(response.text, 'html.parser')
```

Les deux méthodes phares :

Méthode	Ce qu'elle fait	Retourne...
<code>find()</code>	Trouve le premier élément correspondant.	Un objet unique.
<code>find_all()</code>	Trouve tous les éléments correspondants.	Une liste d'objets (itérable).

Cibler par balise et par classe :

```
infobox = soup.find('table', class_ = 'infobox') or soup.find('div',
class_='infobox_v3')
```

Les principales balises html :

<h1> à <h6> : Les titres. **<h1>** est le titre principal, **<h2>** les sous-titres, etc. C'est souvent là qu'on trouve les noms d'articles ou de produits.

<p> : Pour les paragraphes de texte.

<div> : La balise la plus commune. Elle sert de "boîte" pour regrouper d'autres éléments. On l'utilise souvent avec une **class** ou un **id** pour isoler une section (ex: la barre latérale ou une fiche produit).

<table>, <tr>, <td> : Pour les données tabulaires. **<tr>** est une ligne, **<td>** est une cellule.

Ce que je cherche	Balise	Attribut à cibler	Syntaxe BS4
Un titre	<h1>	Aucun (texte)	soup.find('h1').text
Un lien	<a>	href	soup.find('a')['href']
Une image		src	soup.find('img')['src']
Une section	<div>	class_	soup.find('div', class_='content')

Annexe 7 :

On commence par installer Sélénium dans le terminal VSCode.

```
pip install selenium
```

Les importations nécessaires :

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

- Import webdriver : Sélénium fonctionne avec un webdriver qui diffère suivant le moteur de recherche qu'on utilise. J'ai fait seulement avec chrome.
- Import By : Permet de récupérer les données de toutes les façons possibles (Xpath, class_name, link_text ...)
- Import WebDriverWait et expected_conditions : Permet d'éviter les bugs dus au chargement incomplet de la page.

On définit le webdriver :

```
driver = webdriver.Chrome()
```

On donne l'url de la page sur laquelle on souhaite se rendre :

```
driver.get('url')
```

Le code va donc ouvrir une page selon le navigateur avec lequel on travaille et se rendre sur l'url donné. De là, il récupère les infos demandées exclusivement sur cette page qu'il a ouverte, avant de la fermer de lui-même lorsque c'est terminé.

Exemple : (indentation mauvaise parce que ça ne rentre pas sur une ligne)

```
driver.get('https://www.statsfl.com/fr/circuits.aspx')
wait = WebDriverWait(driver, 30)
nom_circuits=wait.until(EC.presence_of_all_elements_located((By.XPATH,
f'//*[@id="ctl00_CPH_Main_GV_Circuit"]/tbody/tr[1]/td[2]')))
for c in nom_circuits:
    print(c.text)
```

Ici, on utilise le Xpath de l'élément de la page qu'on veut scraper.

On peut récupérer les infos d'une page web par différents moyens :

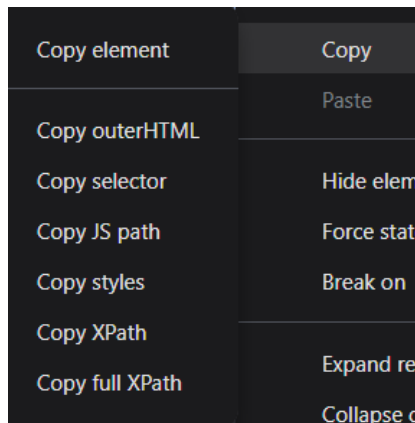
- XPATH
- LINK_TEXT
- CLASS_NAME
- CSS_SELECTOR
- ID
- NAME

- PARTIAL_LINK_TEXT
- TAG_NAME

Lien de la documentation précise pour chaque méthode :

<https://selenium-python.readthedocs.io/locating-elements.html>

Quand on est sur la page web, on copie le XPath de l'élément sélectionné :



On a donc la commande suivante :

```
elements = wait.until( EC.presence_of_all_elements_located((By.XPATH,
'//*[@id="video-title"]')))
```

On récupère une liste de WebElements.

À noter que si on ne souhaite pas attendre que la page soit chargée (pas toujours nécessaire) et que l'on n'utilise pas wait, le code doit être écrit comme ça :

```
driver = webdriver.Chrome()
driver.get('https://www.youtube.com/@Amixem/videos')
elements = driver.find_element(By.XPATH, '//*[@id="video-title"]')
```

On utilise le driver directement sans passer par wait pour récupérer l'élément et on utilise donc la fonction find_element comme avec BeautifulSoup (find_elements fonctionne aussi).]

Pour finir, on imprime le texte de chaque élément de la liste récupérée :

```
for webelem in elements:
    print(webelem.text)
```

Annexe 8 :

driver(driverID, dFirstName, dLastName, dBirthdate, dDeathdate, dCountry, dGender)

Cette table contient les pilotes de courses et d'essai (des saisons de F1 répertoriées) ainsi que les pilotes ayant établi un record du tour d'un circuit.

<u>driverID</u>	identifiant du pilote (clé primaire).
dFirstName	son prénom.
dLastName	son nom (la base ne contient pas les <i>middle names</i> ou les <i>nicknames</i>).
dBirthdate	date de naissance (format 'YYYY-MM-DD').
dDeathdate	date de décès (format 'YYYY-MM-DD') pour un pilote décédé, NULL sinon.
dCountry	pays du pilote (voir la table de codage en dernière page).
dGender	M (male) ou F (female).

grandprix(gpID, gName, #circuitID, gDate, gLaps, gRank)

Cette table contient toutes les épreuves qui ont eu lieu (ou vont avoir lieu) lors des saisons répertoriées.

<u>gpID</u>	identifiant du grandprix (clé primaire, format 'YYYYNN' année/rank).
gName	nom officiel de l'épreuve, dans la langue du pays.
circuitID	identifiant du circuit d'accueil (clé étrangère).
gDate	date et heure (locale) de l'épreuve (format 'YYYY-MM-DD HH :MM :SS'), qui peut être dans le futur pour un grand prix de la saison en cours.
gLaps	nombre de tours à courir lors de l'épreuve.
gRank	rang du grand prix dans la saison.

racedriver(#teamID, #driverID, rSeason, rDriverNb)

Cette table fait le lien entre les pilotes de courses et leurs équipes pour chaque saison.

<u>teamID</u>	identifiant de l'équipe (partie de la clé primaire composite, clé étrangère).
<u>driverID</u>	identifiant du pilote (partie de la clé primaire composite, clé étrangère).
<u>rSeason</u>	saison concernée (format 'YYYY') (partie de la clé primaire composite).
rDriverNb	numéro du pilote (= numéro de la voiture) pour la saison.

Dans les annales récentes, nous n'avons pas trouvé d'exemple d'un pilote ayant couru pour 2 équipes la même saison, mais nous n'avons pas non plus trouvé d'interdiction de le faire dans le règlement... Dans le doute cela est permis dans la BD (d'où la clé composite sur 3 attributs).

standings(#driverID, #gpID, sGrid, sPos, sInc, sPoints, sLaps)

Cette table contient les résultats des courses qui ont eu lieu jusqu'au 21 septembre 2014.

<u>driverID</u>	identifiant du pilote (partie de la clé primaire composite, clé étrangère).
<u>gpID</u>	identifiant du grandprix (partie de la clé primaire composite, clé étrangère).
sGrid	position sur la grille de départ.
sPos	position à l'arrivée, NULL si le pilote n'a pas été classé.
sInc	incident de course (DNF : n'a pas fini la course ; DNS : n'a pas pris le départ ; DQ : disqualifié), ou NULL si aucun incident.
sPoints	points obtenus.
sLaps	nombre de tours entièrement effectués.

team(teamID, tName, tCountry, #twas)

Cette table contient les équipes qui ont participé aux saisons de F1 référencées.

teamID identifiant de l'équipe (clé primaire).
tName nom de l'équipe.
tCountry pays de l'équipe (voir la table de codage en dernière page).
twas NULL si l'équipe a été créée de toute pièce (ou existe depuis la première saison référencée), identifiant de l'ancienne équipe si elle a été créée suite au rachat ou à la transformation d'une équipe existante de la base (clé étrangère).

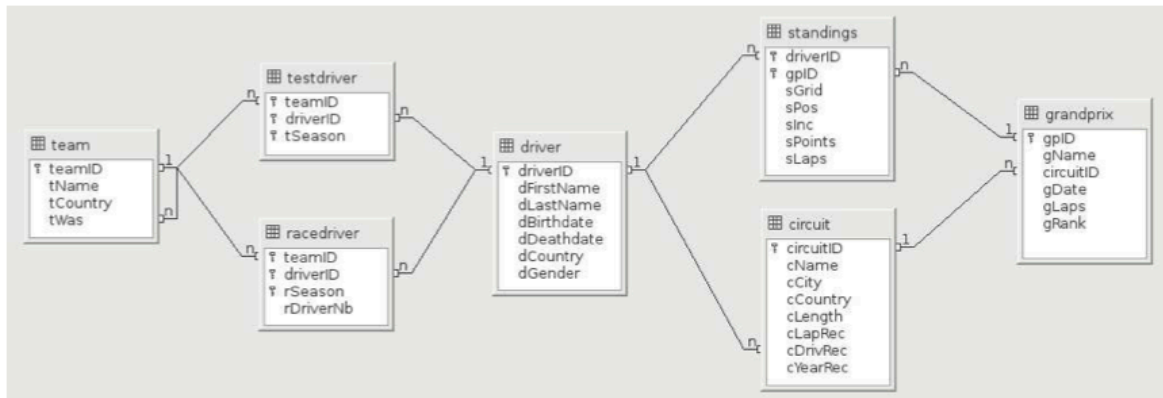
testdriver(#teamID, #driverID, tSeason)

Cette table fait le lien entre les pilotes d'essai et leurs équipes pour chaque saison. Un pilote d'essai peut changer d'équipe en cours de saison (c'est arrivé en 2014).

teamID identifiant de l'équipe (partie de la clé primaire composite, clé étrangère).
driverID identifiant du pilote (partie de la clé primaire composite, clé étrangère).
tSeason saison concernée (format 'YYYY') (partie de la clé primaire composite).

2.3 Schéma

La figure ci-dessous montre les 7 tables de la base et les liens entre clés primaires et clés étrangères.



2.4 Exemples de contenus

circuit							
circuitID	cName	cCity	cCountry	cLength	cLapRec	#cDrivRec	cYearRec
7	Circuit Gilles-Villeneuve	Montréal	Canada	4.361	74.6	27	2004
19	Yas Marina circuit	Yas Marina	Abu Dhabi	5.554	100.3	23	2009
...							

driver						
driverID	dFirstName	dLastName	dBirthdate	dDeathdate	dCountry	dGender
19	Daniel	Ricciardo	1989-07-01	NULL	Australia	M
23	Sebastian	Vettel	1987-07-03	NULL	Germany	M
...						

grandprix					
gpID	gName	#circuitID	gDate	gLaps	gRank
201407	GRAND PRIX DU CANADA	7	2014-06-08 14:00:00	70	7
201419	ETIHAD AIRWAYS ABU DHABI GRAND PRIX	19	2014-11-23 17:00:00	55	19
...					

racedriver			
#teamID	#driverID	rSeason	rDriverNb
4	1	2014	14
2	23	2014	1
...			

standings						
#driverID	#gpID	sGrid	sPos	sInc	sPoints	sLaps
19	201407	6	1	NULL	25	70
20	201407	1	2	NULL	18	70
...						

team			
teamID	tName	tCountry	#twas
2	Red Bull Racing-Renault	Austria	NULL
4	Ferrari	Italy	NULL
...			

tesdriver		
#teamID	#driverID	rSeason
2	29	2014
2	30	2014
...		

2.5 Listes de valeurs

Les valeurs utilisées pour l'attribut `tCountry` de la table `team`, l'attribut `dCountry` de la table `driver` et l'attribut `cCountry` de la table `circuit` sont :

- | | |
|-----------------|-----------------|
| — Abu Dhabi | — Italy |
| — Argentina | — Japan |
| — Australia | — Korea |
| — Austria | — Malaysia |
| — Bahrain | — Mexico |
| — Belgium | — Monaco |
| — Brazil | — Netherlands |
| — Canada | — New Zealand |
| — China | — Portugal |
| — Colombia | — Russia |
| — Denmark | — Singapore |
| — Finland | — South Africa |
| — France | — Spain |
| — Germany | — Sweden |
| — Great Britain | — Switzerland |
| — Hungary | — United States |
| — India | — Venezuela |

Annexe 9 :

Exemple des tables obtenues :

circuit								
circuitID	cName	cCity	cCountry	cLength	cLapRec	cDrivRec	cYearRec	cDates
3	Ain Diab	Casablanca	Morocco	7.603 km	142.5	459	1958	1957-1958
27	Bahrain International Circuit	Sakhir	Bahrain	5.417 km	90.252	600	2004	2004

driver						
driverID	dFirstName	dlastName	dBirthdate	dDeathdate	dCountry	dGender
1	George	Abecassis	1913-03-21	1991-12-18	Royaume-Uni	H
729	Divina	Galica	1944-08-13	NULL	Royaume-Uni	F

grandprix						
gpID	#seasonID	gName	circuitID#	gDate	gLaps	gRank
1	1950	British Grand Prix	302	1950-05-13 00:00:00	70	1
38	1953	German Grand Prix	222	1953-08-02 00:00:00	18	7

racedriver					
#teamID	#driverID	#seasonID	rDriverNB	rTyres	rMotor
124	639	1974	9	GoodYear	Ford Cosworth
69	547	1974	11	GoodYear	Ferrari

standings						
#gpID	#driverID	sPos	sPoints	sGrid	sLaps	sInc
1	220	2	6	2	70	RAS
19	309	14	0	15	51	Technical

team					
teamID	tName	tCountry	tstart	tend	twas
6	Alfa Romeo	Suisse	2019	2023	176
7	Alfa Special	Afrique du Sud	1963	1963	NULL

season				
seasonID	seTyres	seWorldChamp	seConstChamp	seFixedNb
1950	NULL	Giuseppe Farina	NULL	False
2014	Pirelli	Lewis Hamilton	Mercedes	True

Annexe 10 :

Absolument. Sur Mac, Selenium est un excellent choix car il permet de piloter un navigateur (Chrome ou Firefox) pour simuler un comportement humain, ce qui est idéal pour un site comme StatsF1 qui peut avoir des éléments chargés dynamiquement.

Voici ton guide de démarrage rapide "Spécial Mac".

1. Préparation de l'environnement

Ouvre ton terminal et installe les outils nécessaires. Nous allons utiliser Python, qui est le plus simple pour ce genre de tâche.

```
Bash

# Installation de Selenium
pip install selenium

# Installation de webdriver-manager (pour éviter de gérer manuellement les drivers)
pip install webdriver-manager
```

2. Ton premier script de "Scraping"

Voici un squelette de code pour extraire, par exemple, le nom du manufacturier de pneus pour l'année 1961.

```
Python

from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
import time

# 1. Configuration du navigateur (Chrome)
options = webdriver.ChromeOptions()
# options.add_argument('--headless') # Décommente pour ne pas voir la fenêtre s'ouvrir
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=options)

try:
    # 2. Aller sur la page de StatsF1
    driver.get("https://www.statsf1.com/fr/1961.aspx")

    # 3. Attendre un peu que la page charge
    time.sleep(2)

    # 4. Trouver l'élément (Exemple : le texte dans le tableau des pneus)
```

Fonction	Utilisation (Syntaxe)	Rôle / Action
webdriver.Chrome()	<code>driver = webdriver.Chrome(...)</code>	Le Pilote : Ouvre une instance du navigateur Chrome sur ton ordinateur.
get()	<code>driver.get("url")</code>	La Navigation : Ordonne au navigateur d'aller sur une page Web spécifique (équivalent de l'URL).
time.sleep()	<code>time.sleep(secondes)</code>	La Pause : Arrête le script pendant X secondes pour laisser la page (images, tableaux) charger.
find_element()	<code>driver.find_element(By.ID, "nom")</code>	La Recherche Unique : Trouve le premier élément qui correspond au critère. Renvoie une erreur s'il ne trouve rien.
find_elements()	<code>driver.find_elements(By.TAG_NAME, "tr")</code>	La Recherche Multiple : Trouve tous les éléments correspondants et les range dans une liste Python .
.text	<code>variable.text</code>	L'Extraction : Récupère uniquement le texte visible à l'écran (enlève les balises HTML comme <code><td></code> ou <code><a></code>).
.click()	<code>bouton.click()</code>	L'Interaction : Simule un clic de souris sur l'élément sélectionné (bouton, lien, menu).
By.ID	<code>By.ID, "identifiant_unique"</code>	Le Filtre ID : Cible un élément par son identité unique (le moyen le plus sûr en SQL/HTML).
By.TAG_NAME	<code>By.TAG_NAME, "td"</code>	Le Filtre Balise : Cible les éléments par leur type (tous les liens <code>a</code> , toutes les cellules <code>td</code>).

Annexe 11 :

```
# DICTIONNAIRE DE FILIATION (Enfant -> Parent)
dictionnaire_twas = {
    "Stake F1 Team": "Alfa Romeo",
    "Alfa Romeo": "Sauber",
    "Sauber": "BMW Sauber F1 Team",
    "BMW Sauber F1 Team": "Sauber",
    "Alpine F1 Team": "Renault F1 Team",
    "Renault F1 Team": "Lotus F1 Team",
    "Lotus F1 Team": "Renault F1 Team",
    "Benetton Formula": "Toleman",
    "Mercedes Grand Prix": "Brawn GP Formula One Team",
    "Brawn GP Formula One Team": "Honda Racing F1 Team",
    "Honda Racing F1 Team": "British American Racing",
    "British American Racing": "Tyrrell Racing",
    "Red Bull Racing": "Jaguar Racing",
    "Jaguar Racing": "Stewart Grand Prix",
```

Annexe 12 :

Filiation :

- **Sauber** (1993-2005) → **BMW Sauber** (2006-2010) → **Sauber** (2011-2018) → **Alfa Romeo** (2019-2023) → **Kick Sauber** (2024-2025)
- **Audi** (2026-)

Annexe 13 :

Période	Manufacturier	Nombre de Saisons	Type
1961 - 1963	Dunlop	3 saisons	De fait
1987 - 1988	Goodyear	2 saisons	De fait
1992 - 1996	Goodyear	5 saisons	De fait
1999 - 2000	Bridgestone	2 saisons	De fait
2007 - 2010	Bridgestone	4 saisons	Réglementaire
2011 - 2024+	Pirelli	14+ saisons	Réglementaire

Annexe 14 :

```

    "url": "https://en.wikipedia.org/wiki/2006_Chinese_Grand_Prix",
    "raceName": "Chinese Grand Prix",
    "Circuit": {
        "circuitId": "shanghai",
        "url": "https://en.wikipedia.org/wiki/Shanghai_International_Circuit",
        "circuitName": "Shanghai International Circuit",
        "Location": {
            "lat": "31.3389",
            "long": "121.22",
            "locality": "Shanghai",
            "country": "China"
        }
    },
    "date": "2006-10-01",
    "time": "14:00:00Z",
    "FirstPractice": {
        "date": "2006-09-29"
    },
    "SecondPractice": {
        "date": "2006-09-29"
    },
    "ThirdPractice": {
        "date": "2006-09-30"
    },
    "Qualifying": {
        "date": "2006-09-30"
    }
},
{
    "season": "2006",
    "round": "17",
    "url": "https://en.wikipedia.org/wiki/2006_Japanese_Grand_Prix",
    "raceName": "Japanese Grand Prix",
    "Circuit": {
        "circuitId": "suzuka",
        "url": "https://en.wikipedia.org/wiki/Suzuka_International_Racing_Course",
        "circuitName": "Suzuka Circuit",
        "Location": {
            "lat": "34.8431"
        }
    }
}
```

Annexe 15 :

```
{
  "MRData": {
    "xmlns": "",
    "series": "f1",
    "url": "http://api.jolpi.ca/ergast/f1/2021/9/results/",
    "limit": "30",
    "offset": "0",
    "total": "20",
    "RaceTable": {
      "season": "2021",
      "round": "9",
      "Races": [
        {
          "season": "2021",
          "round": "9",
          "url": "http://en.wikipedia.org/wiki/2021_Austrian_Grand_Prix",
          "raceName": "Austrian Grand Prix",
          "Circuit": {
            "circuitId": "red_bull_ring",
            "url": "http://en.wikipedia.org/wiki/Red_Bull_Ring",
            "circuitName": "Red Bull Ring",
            "Location": {
              "lat": "47.2197",
              "long": "14.7647",
              "locality": "Spielberg",
              "country": "Austria"
            }
          }
        },
        {
          "date": "2021-07-04",
          "time": "13:00:00Z",
          "Results": [
            {
              "number": "33",
              "position": "1",
              "positionText": "1",
              "points": "26",
              "Driver": {
                "driverId": "max_verstappen",
                "permanentNumber": "33",
                "code": "VER",
                "url": "http://en.wikipedia.org/wiki/Max_Verstappen",
                "givenName": "Max",
                "familyName": "Verstappen",
                "dateOfBirth": "1997-09-30",
                "nationality": "Dutch"
              },
              "Constructor": {
                "constructorId": "red_bull",
                "url": "http://en.wikipedia.org/wiki/Red_Bull_Racing",
                "name": "Red Bull",
                "nationality": "Austrian"
              },
              "grid": "1",
              "laps": "71",
              "status": "Finished",
              "Time": {
                "millis": "5034543",
                "time": "1:23:54.543"
              },
              "FastestLap": {
                "rank": "1",
                "lap": "62",
                "Time": {
```